```
/* ======== 文件: server/index.js ======== */
'use strict';
/**
 * HTTP 服务 (原生 http, 零依赖)
 *  - 静态: 把项目根目录当静态根, 原型页面直接可访问 (推荐直接从本服务打开页面 → 同源, 无跨域)
 *  - API : /api/** 走 JSON 路由, 按「公开 / 练习 / 管理」三档校验身份
 *  - CORS: 默认只放行同源 + 本机来源 + file://, 不再使用通配 *
 *  - 鉴权: 见 server/auth.js (AUTH_MODE=open|enforce)
 */

const http = require('http');
const fs = require('fs');
const path = require('path');
const url = require('url');

const config = require('./config');
const dbm = require('./db');
const svc = require('./service');

/**
 * baseUrl 是否指向本机 —— 即"用的是内置 Mock 模型"而不是真实大模型。
 *
 * 抽成函数是因为 `/status` 的 note 与启动横幅两处都要判断;
 * 两处各写一遍迟早会不一致 (而"到底在用哪个模型"恰恰是最不能含糊的一件事)。
 */
function isLocalLlm() {
  let host = '';
  try { host = new URL(config.llm.baseUrl).hostname; } catch (e) { host = ''; }
  return host === '127.0.0.1' || host === 'localhost' || host === '::1';
}
const gold = require('./goldset');
const ai = require('./ai');
const rubric = require('./rubric');
const voice = require('./voice');
const auth = require('./auth');
const backup = require('./backup');

const MIME = {
  '.html': 'text/html; charset=utf-8',
  '.js': 'text/javascript; charset=utf-8',
  '.css': 'text/css; charset=utf-8',
  '.json': 'application/json; charset=utf-8',
  '.png': 'image/png', '.jpg': 'image/jpeg', '.jpeg': 'image/jpeg',
  '.svg': 'image/svg+xml', '.ico': 'image/x-icon',
  '.woff2': 'font/woff2', '.mp3': 'audio/mpeg', '.m4a': 'audio/mp4',
  // P11: 浏览器录音/测试样本的音频类型 (缺了会以 octet-stream 下发,
  // 页面 fetch 后 blob.type 丢掉 audio/ 前缀, /asr 上传链路跟着断)
  '.wav': 'audio/wav', '.webm': 'audio/webm', '.ogg': 'audio/ogg'
};

```

```
/* ------------------------------------------------------------------ *
 * 小工具
 * ------------------------------------------------------------------ */
function sendJSON(res, code, obj) {
  const body = JSON.stringify(obj);
  res.writeHead(code, {
    'Content-Type': 'application/json; charset=utf-8',
    'Content-Length': Buffer.byteLength(body),
    'Cache-Control': 'no-store'
  });
  res.end(body);
}

function readBody(req, raw) {
  return new Promise((resolve, reject) => {
    let chunks = [], size = 0;
    req.on('data', (c) => {
      size += c.length;
      if (size > 8 * 1024 * 1024) { reject(Object.assign(new Error('请求体过大'), { status: 413 }
      chunks.push(c);
    });
    req.on('end', () => {
      const buf = Buffer.concat(chunks);
      // raw 模式（P11 /asr 音频）：原样返回 Buffer，不做 utf8/JSON 解析
      if (raw) return resolve(buf.length ? buf : Buffer.alloc(0));
      const s = buf.toString('utf8');
      if (!s) return resolve({});
      try { resolve(JSON.parse(s)); }
      catch (e) { reject(Object.assign(new Error('请求体不是合法 JSON'), { status: 400 })); }
    });
    req.on('error', reject);
  });
}

/* 静态资源白名单 + 目录黑名单（P33，上线前安全）
 *
 * 原实现把项目根目录下**任意文件**都发出去。实测（本地就跑）：
 *   GET /key.txt                  → 200  ← LLM API Key 明文
 *   GET /server/.env.json         → 200  ← 同上
 *   GET /server/store/training.db → 200  ← 全部学员姓名/成绩/辅导记录
 * 本地无所谓，一旦发布到公网就是"把钥匙挂在门上"。
 *
 * 双保险：只发前端真正需要的类型（扩展名白名单），后端/文档/隐藏目录一律不发。
 * 用 404 而不是 403 —— 不透露这些文件是否存在。
 */
const STATIC_EXT = new Set([
  '.html', '.js', '.css', '.json', '.map',
  '.png', '.jpg', '.jpeg', '.svg', '.ico', '.webp', '.gif',
  '.wav', '.webm', '.mp3', '.mp4',
  '.woff', '.woff2', '.ttf'
```

```
]);
const DENY_DIRS = new Set(['server', 'docs', 'node_modules', '.git', '.workbuddy', 'refs', 'tool

function staticDenied(rel) {
  const segs = rel.split('/').filter(Boolean);
  for (const seg of segs) {
    if (seg.startsWith('.')) return true;                    // .env.json / .git / .workbuddy
    if (DENY_DIRS.has(seg.toLowerCase())) return true;       // 后端与文档目录
  }
  return !STATIC_EXT.has(path.extname(rel).toLowerCase());   // .db / .log / .md / .txt / .pdf
}

function serveStatic(req, res, pathname) {
  let rel = decodeURIComponent(pathname);
  if (rel === '/' || rel === '') rel = '/index.html';
  if (staticDenied(rel)) {
    res.writeHead(404, { 'Content-Type': 'text/plain; charset=utf-8' });
    res.end('404 Not Found');
    return;
  }
  // 防目录穿越
  const target = path.normalize(path.join(config.root, rel));
  if (!target.startsWith(config.root)) { res.writeHead(403); res.end('Forbidden'); return; }
  fs.stat(target, (err, st) => {
    if (err || !st.isFile()) {
      res.writeHead(404, { 'Content-Type': 'text/plain; charset=utf-8' });
      res.end('404 Not Found: ' + rel);
      return;
    }
    const ext = path.extname(target).toLowerCase();
    res.writeHead(200, {
      'Content-Type': MIME[ext] || 'application/octet-stream',
      'Content-Length': st.size
    });
    fs.createReadStream(target).pipe(res);
  });
}

/* ------------------------------------------------------------------ *
 * CORS：来源白名单（不再通配）
 *   放行条件（任一）：
 *     - 无 Origin（curl / 服务端调用 / 同源简单请求）
 *     - Origin === 'null' 且允许 file:// 页面
 *     - 在 CORS_ORIGINS 显式清单里
 *     - 本机来源（127.0.0.1 / localhost / [::1]，任意端口）且 CORS_ALLOW_LOCAL 打开
 *     - 与请求 Host 同源
 * ------------------------------------------------------------------ */
function originAllowed(req) {
  const o = req.headers.origin;
  if (!o) return true;
```

```
  if (o === 'null') return config.cors.allowFile;
  if (config.cors.origins.indexOf(o) !== -1) return true;
  try {
    const u = new URL(o);
    if (config.cors.allowLocal &&
        (u.hostname === '127.0.0.1' || u.hostname === 'localhost' || u.hostname === '::1')) retu
    const host = req.headers.host || '';
    if (host && u.host === host) return true;
  } catch (e) { /* ignore */ }
  return false;
}

function applyCors(req, res) {
  const o = req.headers.origin;
  if (!o) return true;
  if (!originAllowed(req)) return false;
  res.setHeader('Access-Control-Allow-Origin', o);
  res.setHeader('Vary', 'Origin');
  res.setHeader('Access-Control-Allow-Credentials', 'true');
  return true;
}

/* ---------------------------------------------------------------- *
 * 路由表：method + path 正则 → handler(ctx)
 *   ctx = { req, res, params, query, body, user, authSource }
 *   权限档位 level：
 *     public   —— 无需身份（探活 / 登录 / Rubric）
 *     practice —— 练习通道，允许匿名（学员端必须「没登录也能练」）
 *     staff    —— 管理类接口，必须可解析出身份（enforce 模式下必须带 token）
 * ---------------------------------------------------------------- */
const routes = [];
function on(method, pattern, handler, level, opts) {
  routes.push({ method, pattern, handler, level: level || 'staff', rawBody: !!(opts && opts.rawB
}

/* ---- 鉴权 ---- */
on('POST', /^\/api\/v1\/auth\/login$/, async (c) => {
  expectFields(c.body, ['account', 'password']);
  return auth.login(c.body.account, c.body.password, { ip: clientIp(c.req) });
}, 'public');

on('GET', /^\/api\/v1\/auth\/me$/, async (c) => ({
  user: auth.publicUser(c.user), source: c.authSource, auth: auth.modeInfo()
}));

on('POST', /^\/api\/v1\/auth\/password$/, async (c) => {
  expectFields(c.body, ['oldPassword', 'newPassword']);
  return auth.changePassword(c.user.id, c.body.oldPassword, c.body.newPassword);
});

```

```
on('GET', /^\/api\/v1\/auth\/accounts$/, async () => ({
  items: auth.listAccounts(), auth: auth.modeInfo()
}));

/* ---- 状态 ---- */
on('GET', /^\/api\/v1\/status$/, async () => {
  const aiStat = await ai.aiStatus();
  const asrOk = await voice.asrOnline();
  return {
    ok: true,
    service: '新员工培训系统 · AI 服务层',
    version: 'p7',
    db: path.basename(config.dbFile),
    dbSync: dbm.pragmas(),
    host: config.host,
    // 语音识别（P11）：本地 SenseVoiceSmall。如实报在线状态——
    // 不在线时学员端自动回退键盘，别把「没起」说成「可用」。
    voice: {
      asrOnline: asrOk,
      asrBase: config.voice.asrBase,
      model: 'SenseVoiceSmall',
      note: asrOk ? '本地 FunASR · SenseVoiceSmall 在线'
        : '本地 ASR 服务未连接（语音输入不可用，学员端自动回退键盘输入）'
    },
    llm: {
      configured: config.llm.enabled,
      baseUrl: config.llm.baseUrl,
      model: config.llm.model,
      note: (() => {
        if (!config.llm.enabled) return '未配置 LLM_API_KEY，自动降级为关键词引擎 + 剧本';
        // ⚠ 别把「配了 Key」直接说成「走真实模型」：本机 mock 也满足"已配置"，
        // 这样写会让人（包括自己）误以为已在用真实大模型。界面上不许在这种事上说谎。
        return isLocalLlm()
          ? '已配置 Key，但 baseUrl 指向本机 —— 当前是内置 Mock 模型，不是真实大模型（仅用于验证全链路
          : '已配置 Key，走真实模型';
      })()
    },
    live: aiStat && aiStat.live ? aiStat.live : { ok: !!config.llm.enabled, mode: config.llm.ena
    auth: auth.modeInfo(),
    cors: { origins: config.cors.origins, allowLocal: config.cors.allowLocal, allowFile: config.
    rubricVersion: config.rubricVersion,
    stats: svc.stats()
  };
}, 'public');

on('GET', /^\/api\/v1\/ai\/status$/, async () => ai.aiStatus(), 'public');
on('GET', /^\/api\/v1\/rubric$/, async () => rubric.currentRubric(), 'public');

/* ---- 语音识别（P11）：本地 FunASR · SenseVoice-Small ----
 * 档位刻意与练习通道同为 practice：学员没登录也要能用语音（核心约束）。
```

```
 * 请求体是音频二进制（Content-Type: audio/*），由分发层按 raw 模式读入。 */
on('POST', /^\/api\/v1\/asr$/, async (c) => {
  if (!Buffer.isBuffer(c.body) || !c.body.length) {
    throw Object.assign(new Error('请用 Content-Type: audio/* 上传音频二进制'), { status: 400 });
  }
  const out = await voice.transcribe(c.body);
  if (!out.ok) throw Object.assign(new Error(out.error), { status: out.status || 500 });
  return { text: out.text, inferMs: out.inferMs };
}, 'practice', { rawBody: true });

/* ---- 场景库 ---- */
on('GET', /^\/api\/v1\/scenes$/, async (c) => ({ items: svc.listScenes(c.query) }));
// ⚠ 必须排在 /scenes/:id 之前：路由按**注册顺序**首个匹配，
//    否则 'published' 会被当成场景 id 走进详情分支。
//    练习档（匿名可读）：学员必须没登录也能取到场景库，且只拿得到已发布场景。
on('GET', /^\/api\/v1\/scenes\/published$/, async () => ({
  items: svc.listPublishedScenes(), version: 'published'
}), 'practice');
/* 场景草稿生成（P31）：粘贴真实客户对话 → 可编辑的场景草稿。
 * ⚠ 必须排在 /scenes/:id 之前（铁律 13：路由按注册顺序首个匹配）。
 * 只生成草稿、**不落库**—运营在编辑器改完仍要走提交审核，绕过不了 canAssign 闸门。 */
on('POST', /^\/api\/v1\/scenes\/draft$/, async (c) => svc.generateSceneDraft(c.body, c.user.id))
on('GET', /^\/api\/v1\/scenes\/([^/]+)$/, async (c) => {
  const s = svc.getScene(c.params[0]);
  if (!s) throw Object.assign(new Error('场景不存在'), { status: 404 });
  return s;
});
on('POST', /^\/api\/v1\/scenes$/, async (c) => svc.saveScene(c.body, c.user.id));
/* 复制场景为草稿（P14）：运营「基于现有改一版」的一键入口，复制品强制 draft */
on('POST', /^\/api\/v1\/scenes\/([^/]+)\/duplicate$/, async (c) =>
  svc.duplicateScene(c.params[0], c.user && c.user.id));
on('PUT', /^\/api\/v1\/scenes\/([^/]+)$/, async (c) =>
  svc.saveScene(Object.assign({}, c.body, { id: c.params[0] }), c.user.id));
on('DELETE', /^\/api\/v1\/scenes\/([^/]+)$/, async (c) => svc.deleteScene(c.params[0], c.user.id

on('GET', /^\/api\/v1\/scenes\/([^/]+)\/versions$/, async (c) => ({ items: svc.listVersions(c.pa
on('POST', /^\/api\/v1\/scenes\/([^/]+)\/rollback$/, async (c) => {
  expectFields(c.body, ['v']);
  return svc.rollback(c.params[0], c.body.v, c.user.id);
});

/* ---- 审批流 ---- */
on('POST', /^\/api\/v1\/scenes\/([^/]+)\/submit$/, async (c) => svc.submitReview(c.params[0], c.
on('POST', /^\/api\/v1\/scenes\/([^/]+)\/approve$/, async (c) => svc.approveScene(c.params[0], c
on('POST', /^\/api\/v1\/scenes\/([^/]+)\/reject$/, async (c) => svc.rejectScene(c.params[0], c.b
on('POST', /^\/api\/v1\/scenes\/([^/]+)\/offline$/, async (c) => svc.offlineScene(c.params[0], c

/* ---- 对话（练习通道：允许匿名，学员端必须没登录也能练） ---- */
on('POST', /^\/api\/v1\/session\/([^/]+)\/turn$/, async (c) =>
  svc.turn(Object.assign({}, c.body, { sessionId: c.params[0] })), 'practice');
```

```
on('POST', /^\/api\/v1\/session\/([^/]+)\/finish$/, async (c) =>
  svc.finish(Object.assign({}, c.body, { sessionId: c.params[0] })), 'practice');

/* ---- 任务（P4）：学员端只读自己的；下发/改期/结束/删除需 staff 身份 ---- */
on('GET', /^\/api\/v1\/tasks$/, async (c) => {
  // 学员端：允许匿名读自己的任务清单（不登录也要能能练）
  if (c.query.learnerId) return svc.listTasks(c.query, null, true);
  if (!c.user) throw Object.assign(new Error('未登录或登录状态已失效'), { status: 401 });
  return svc.listTasks(c.query, c.user);
}, 'practice');

on('GET', /^\/api\/v1\/tasks\/([^/]+)$/, async (c) => {
  const t = svc.getTask(c.params[0]);
  if (!t) throw Object.assign(new Error('任务不存在'), { status: 404 });
  return t;
});
on('POST', /^\/api\/v1\/tasks$/, async (c) => svc.saveTask(c.body, c.user.id));
on('PUT', /^\/api\/v1\/tasks\/([^/]+)$/, async (c) =>
  svc.saveTask(Object.assign({}, c.body, { id: c.params[0] }), c.user.id));
on('POST', /^\/api\/v1\/tasks\/([^/]+)\/status$/, async (c) => {
  expectFields(c.body, ['status']);
  return svc.setTaskStatus(c.params[0], c.body.status, c.user.id);
});
on('DELETE', /^\/api\/v1\/tasks\/([^/]+)$/, async (c) => svc.deleteTask(c.params[0], c.user.id))

/* ---- 教练动作（P5）：辅导记录 / 对话质检 ----
 * 这两件事是**带教主管的本职**，所以不卡 can_edit / can_review，
 * 只要求可解析出身份（staff），范围校验在 service 层按数据可见范围做。 */
/* 学员也能读自己的带教留言（P21）：辅导要闭环，学员必须看得到教练写了什么 */
on('GET', /^\/api\/v1\/coach-notes$/, async (c) => {
  requireLearnerIdForAnonymous(c);
  return svc.listCoachNotes(c.query, c.user);
}, 'practice');
on('POST', /^\/api\/v1\/coach-notes$/, async (c) => svc.saveCoachNote(c.body, c.user.id));
/* 学员回复带教留言（P29）：辅导闭环的最后一步。
 * 走独立路由而不是把 /coach-notes 放开给 practice —— 后者是 staff 写入接口，
 * 匿名放开等于任何人可代任意学员写辅导记录。这里只允许"回复已有留言"，
 * 归属由服务端按 replyTo 反查校验，越权面收窄到最小。 */
on('POST', /^\/api\/v1\/learner\/coach-notes\/reply$/, async (c) => svc.replyCoachNote(c.body),
on('DELETE', /^\/api\/v1\/coach-notes\/([^/]+)$/, async (c) => svc.deleteCoachNote(c.params[0],
on('GET', /^\/api\/v1\/qc$/, async (c) => ({ items: svc.listQc(c.user) }));
on('POST', /^\/api\/v1\/records\/([^/]+)\/qc$/, async (c) => {
  expectFields(c.body, ['state']);
  return svc.setQc(c.params[0], c.body.state, c.body.note, c.user.id);
});
on('DELETE', /^\/api\/v1\/records\/([^/]+)\/qc$/, async (c) => svc.clearQc(c.params[0], c.user.i
// 逐轮对话（P7）：列表接口刻意不带正文，页面按需取这一条
/* 待建场景池（P12）：教练标记 → 运营认领（内容供给闭环） */
on('GET', /^\/api\/v1\/scene-ideas$/, async (c) => svc.listIdeas(c.query));
on('POST', /^\/api\/v1\/scene-ideas$/, async (c) => svc.createIdea(c.body, c.user && c.user.id))
```

```
on('DELETE', /^\/api\/v1\/scene-ideas\/([^/]+)$/, async (c) =>
  svc.deleteIdea(c.params[0], c.user && c.user.id));
on('POST', /^\/api\/v1\/scene-ideas\/([^/]+)\/status$/, async (c) =>
  svc.setIdeaStatus(c.params[0], c.body && c.body.status, c.user && c.user.id));

/* 人工修正分数（P12）：AI 分可被带教主管推翻，修正值与原分并存（审计留痕） */
on('PATCH', /^\/api\/v1\/records\/([^/]+)$/, async (c) =>
  svc.setManualScore(c.params[0], c.body && c.body.score, c.body && c.body.reason, c.user && c.u

on('GET', /^\/api\/v1\/records\/([^/]+)\/turns$/, async (c) => svc.listTurns(c.params[0], c.user
// 必须排在 /records/:id/qc 之后：前者带子路径，先匹配更具体的
on('DELETE', /^\/api\/v1\/records\/([^/]+)$/, async (c) => svc.deleteRecord(c.params[0], c.user.

/* ---- 数据查询（全部按登录身份的数据范围过滤） ----
 * 五个列表接口统一分页口径（P9）：不带 limit/offset 返回全量（audit 默认最近 100 条）；
 * 带则校验 limit∈[1,500] / offset>=0，返回 { items, total, limit, offset, hasMore }。 */
/* 学员也要能读自己的练习记录（P30）。
 * ⚠ 无凭据请求在 open 模式下会被 auth.resolve 兜底成 admin（既有设计，方便演示），
 * 于是"匿名"拿到的其实是全量权限——学员端恰好走无凭据路径，前端一旦漏传 learnerId，
 * 就会把全库成绩渲到某个学员屏幕上。这里加一道硬闸：无凭据必须指明学员。 */
function requireLearnerIdForAnonymous(c) {
  if (c.authSource === 'fallback' && !c.query.learnerId) {
    throw Object.assign(new Error('无凭据请求需指定 learnerId（学员只能读自己）'), { status: 400 })
  }
}
on('GET', /^\/api\/v1\/records$/, async (c) => {
  requireLearnerIdForAnonymous(c);
  return svc.listRecords(c.query, c.user);
}, 'practice');
/* 学员读自己某条成绩的逐轮对话：匿名 practice 档，按记录归属校验 */
on('GET', /^\/api\/v1\/learner\/records\/([^/]+)\/turns$/,
  async (c) => svc.learnerTurns(c.params[0], c.query.learnerId), 'practice');
/* 学员批量导入（P13）：CSV → learners，零依赖（Excel 另存为 CSV 即可） */
on('DELETE', /^\/api\/v1\/admin\/learners\/([^/]+)$/, async (c) =>
  svc.deleteLearner(c.params[0], c.user && c.user.id));
on('POST', /^\/api\/v1\/admin\/learners\/import$/, async (c) =>
  svc.importLearners(c.body, c.user && c.user.id));

on('GET', /^\/api\/v1\/learners$/, async (c) => svc.listLearners(c.query, c.user));
on('GET', /^\/api\/v1\/users$/, async () => ({ items: svc.listUsers() }));
on('GET', /^\/api\/v1\/audit$/, async (c) => svc.listAuditLog(c.query, c.user));

/* ---- 金标集与校准 ---- */
on('GET', /^\/api\/v1\/goldset$/, async () => ({ items: gold.listGold() }));
on('POST', /^\/api\/v1\/goldset\/import$/, async (c) => gold.importGold(c.body.items || [], c.us
on('POST', /^\/api\/v1\/goldset\/calibrate$/, async (c) => gold.calibrate(c.body || {}));
on('GET', /^\/api\/v1\/goldset\/runs$/, async (c) => ({ items: gold.listRuns(Number(c.query.limi

/* ---- 运维：备份 ---- */
on('GET', /^\/api\/v1\/admin\/backup$/, async () => ({ items: backup.list(), keep: backup.keep,
```

```
on('POST', /^\/api\/v1\/admin\/backup$/, async (c) => backup.create((c.user && c.user.name) || '
/* 清理「无成绩关联」的历史会话（P11 巡检补齐）：学员中途退出的练习、自测/探针调用
 * 都会建 session 但永远没有 record——长期运行会累积。只删无成绩关联的，已完成练习无论多老都保留。 */
on('POST', /^\/api\/v1\/admin\/cleanup-sessions$/, async (c) =>
  svc.cleanupSessions(c.body && c.body.maxAgeHours, c.user && c.user.id));

/* ---------------------------------------------------------------- *
 * 入参校验小工具
 * ---------------------------------------------------------------- */
function expectFields(body, fields) {
  const miss = fields.filter((f) => body[f] === undefined || body[f] === null || body[f] === '')
  if (miss.length) throw Object.assign(new Error('缺少必填字段：' + miss.join('、')), { status: 40
}

function clientIp(req) {
  return (req.headers['x-forwarded-for'] || '').split(',')[0].trim() ||
    (req.socket && req.socket.remoteAddress) || '';
}

/* ---------------------------------------------------------------- *
 * 请求分发
 * ---------------------------------------------------------------- */
function checkLevel(level, idRes) {
  if (level !== 'staff') return null;
  if (idRes.error) return idRes.error;
  if (!idRes.user) {
    return Object.assign(new Error(
      config.auth.mode === 'enforce' ? '该接口需要登录（AUTH_MODE=enforce）' : '需要登录'), { statu
  }
  return null;
}

const server = http.createServer(async (req, res) => {
  const parsed = url.parse(req.url, true);
  const pathname = parsed.pathname;

  // 预检
  if (req.method === 'OPTIONS') {
    if (!originAllowed(req)) { res.writeHead(403); return res.end(); }
    applyCors(req, res);
    res.setHeader('Access-Control-Allow-Methods', 'GET,POST,PUT,DELETE,OPTIONS');
    res.setHeader('Access-Control-Allow-Headers', 'Content-Type,Authorization,X-User-Id');
    res.setHeader('Access-Control-Max-Age', '600');
    res.writeHead(204);
    return res.end();
  }
  const corsOk = applyCors(req, res);

  if (!pathname.startsWith('/api/')) return serveStatic(req, res, pathname);
  if (!corsOk) return sendJSON(res, 403, { ok: false, error: '来源不被允许：' + req.headers.origi
```

```
451
452   for (const r of routes) {
453     if (r.method !== req.method) continue;
454     const m = pathname.match(r.pattern);
455     if (!m) continue;
456     try {
457       // 音频上传按二进制读，不走 JSON 解析（P11 /asr）：
458       // 路由声明 rawBody 优先（/asr 必然收音频），Content-Type 前缀兜底
459       const isAudio = r.rawBody || /^audio\//.test(String(req.headers['content-type'] || ''));
460       const body = (req.method === 'GET' || req.method === 'DELETE') ? {} : await readBody(req,
461
462       // 身份解析：Authorization: Bearer 优先；open 模式下兼容 X-User-Id / body.userId
463       const idRes = auth.resolve(req, Buffer.isBuffer(body) ? {} : body, parsed.query);
464       const denied = checkLevel(r.level, idRes);
465       if (denied) throw denied;
466
467       // 以解析结果为准，避免"请求体自称身份"覆盖 token 身份
468       //  （音频 body 是 Buffer，不注入 userId —— 内容一个字节都不能动）
469       if (idRes.user && !Buffer.isBuffer(body)) {
470         body.userId = idRes.user.id;
471         if (!parsed.query.userId) parsed.query.userId = idRes.user.id;
472       }
473
474       const out = await r.handler({
475         req, res, params: m.slice(1), query: parsed.query, body,
476         user: idRes.user, authSource: idRes.source
477       });
478       return sendJSON(res, 200, { ok: true, data: out });
479     } catch (e) {
480       const code = e.status || 500;
481       if (code >= 500) console.error('[API]', pathname, e);
482       return sendJSON(res, code, {
483         ok: false, error: e.message,
484         validate: e.validate || undefined
485       });
486     }
487   }
488   sendJSON(res, 404, { ok: false, error: '接口不存在：' + req.method + ' ' + pathname });
489 });
490
491 /* ---------------------------------------------------------------- *
492  * 启动
493  * ---------------------------------------------------------------- */
494 function start(opts) {
495   opts = opts || {};
496   const seed = dbm.seedIfEmpty();
497   if (seed.seeded) {
498     console.log(`[db] 已初始化：${seed.scenes} 场景 / ${seed.learners} 学员 / ${seed.tasks} 任务 /
499   }
500   // 老库升级（如 P4→P5）时补了演示内容也要说一声，
```

```
// 否则"教练台怎么突然有数据了 / 怎么还是空的"没人能判断
const up5 = dbm.MIGRATED_P5;
if (up5 && !up5.contentSkipped && (up5.coachNotes || up5.qcMarks)) {
  console.log(`[db] P5 升级补内容：辅导记录 ${up5.coachNotes} 条 / 质检结论 ${up5.qcMarks} 条`);
}
// P7：老库的演示成绩回填逐轮对话 —— 不说一声的话，
// 「教练台怎么突然能点开对话了」同样没人能判断
const up7 = dbm.MIGRATED_P7;
if (up7 && up7.dialogues) {
  console.log(`[db] P7 升级补逐轮对话：${up7.dialogues} 条成绩`);
}
// 账号初始化：给没有口令的用户分配账号与初始口令（首次启动会打印一份）
const created = auth.ensureAccounts(!!opts.resetPasswords);

// 启动自清（P11 优化）：删掉 7 天前「无成绩关联」的会话。
// 学员中途退出的练习、探针调用都会留这类过程记录，长期运行会累积；
// 只动无成绩的（已完成练习无论多老都保留），失败不影响启动。
try {
  const swept = svc.sweepStaleSessions(24 * 7);
  if (swept.removedSessions) {
    console.log(`[db] 已自动清理 7 天前无成绩历史会话：${swept.removedSessions} 个（逐轮 ${swept.r
  }
} catch (e) {
  console.warn('[db] 启动自清历史会话失败（不影响启动）：' + e.message);
}

server.listen(config.port, config.host, async () => {
  const lan = config.host === '0.0.0.0';
  console.log('');
  console.log('  新员工培训系统 · AI 服务层已启动');
  console.log('  ————————————————————————————————————————');
  console.log(`  服务地址   http://127.0.0.1:${config.port}   ${lan ? '（已监听 0.0.0.0，内网可访
  console.log(`  接口前缀   /api/v1`);
  console.log(`  数据库     ${config.dbFile}`);
  const pgm = dbm.pragmas();
  console.log(`  落盘级别   ${pgm.journalMode || '?'} + synchronous=${pgm.synchronousName}` +
    (pgm.synchronousName === 'FULL' ? '（每次提交 fsync，写吞吐会掉到个位数，' + '如需高性能请去掉
  console.log(`  备份目录   ${config.backupDir}（保留最近 ${backup.keep} 份）`);
  console.log(`  大模型     ${config.llm.enabled
    ? `已配置（${config.llm.model} @ ${config.llm.baseUrl}）` + (isLocalLlm() ? '← 指向本机，是
    : '未配置 Key —— 自动降级为关键词引擎 + 剧本'}`);
  console.log(`  Rubric     ${config.rubricVersion}`);
  const asrOk0 = await voice.asrOnline();
  console.log(`  语音识别   ${asrOk0 ? '本地 FunASR · SenseVoiceSmall 在线（voice/asr-server.py
    : '本地 ASR 未连接 —— 语音输入不可用，学员端自动回退键盘（不影响其它功能）'}`);
  console.log(`  鉴权模式   ${config.auth.mode}${config.auth.mode === 'open' ? '（未带 token 时[
  console.log('  ————————————————————————————————————————');
  if (created.length) {
    console.log('  初始账号（请登录后立即修改口令；服务端不保存明文）');
    created.forEach((a) => {
```

```
        console.log(`    ${a.account.padEnd(12)} ${a.password.padEnd(12)} ${a.name} (${a.role}) `
      });
      console.log('');
    }
    console.log('  页面入口（推荐从此地址打开，天然同源、无跨域）');
    console.log(`    学员端      http://127.0.0.1:${config.port}/仿真培训原型-微信风格.html`);
    console.log(`    管理后台    http://127.0.0.1:${config.port}/管理后台-仿真培训.html`);
    console.log(`    教练工作台 http://127.0.0.1:${config.port}/教练工作台-带教主管.html`);
    console.log('');
  });
}

if (require.main === module) start();

module.exports = { server, start, routes };


/* ======== 文件: server/service.js ======== */
'use strict';
/**
 * 业务服务层
 *  - 会话（session / turn / finish）：一轮对话 → 评分 → 落库 Records
 *  - 场景库（CRUD / 版本 / 审批 / 权限校验 / 审计）
 *  - 金标集（导入 / 校准跑批 / 一致性报告）
 *
 * 所有写操作都会写 audit_log。
 */

const dbm = require('./db');
const { db, log, sceneRow, nowStamp, today, J, P } = dbm;
const ai = require('./ai');
const config = require('./config');
/* 用户/权限/可见范围已下沉到 lib/core.js（P22）。
 * 这里解构出来继续用，外部调用方式完全不变。 */
const {
  getUser, requireUser, assertCanEdit, assertCanReview,
  scopeOf, scopedLearnerIds, inScope,
  pageOf, pageMeta, assertLearnerInScope, assertRecordScope
} = require('./lib/core');

/* ============================================================== *
 * 用户 / 权限 / 可见范围 —— 已移至 lib/core.js（P22）
 * ============================================================== */
/* 任务模块已下沉到 lib/tasks.js（P22） */
const {
  assertCanAssignTask, nextTaskCode, assigneesOf, taskRow, getTask,
  taskProgressOf, dueOf, listTasks, saveTask, TASK_STATUS, setTaskStatus, deleteTask
} = require('./lib/tasks');

/* 场景模块已下沉到 lib/scenes.js（P22） */
```

```
const {
  listScenes, sceneForLearner, listPublishedScenes, auditOf, countVersions, getScene,
  canAssign, validateScene, nextSceneSeq, saveScene, duplicateScene, deleteScene,
  listVersions, rollback, submitReview, approveScene, rejectScene, offlineScene
} = require('./lib/scenes');

/* ================================================================== *
 * 会话：一轮对话
 * ================================================================== */
/* 学员模块已下沉到 lib/learners.js (P23)  */
const { listLearners, importLearners, deleteLearner } = require('./lib/learners');

/* 会话与成绩模块已下沉到 lib/records.js (P23)  */
const {
  getOrCreateSession, turnRow, turn, evidenceTurn, finish,
  listRecords, listTurns, learnerTurns, deleteRecord
} = require('./lib/records');

function listUsers() {
  // 注意：绝不外泄 salt / password_hash
  return db.prepare('SELECT id,name,staff_name,account,role,dept,can_edit,can_review,data_scope,
    .map((r) => ({
      id: r.id, name: r.name, staffName: r.staff_name || '', account: r.account || '', role: r.r
      canEdit: !!r.can_edit, canReview: !!r.can_review,
      scope: scopeOf(r).kind, scopeRef: r.scope_ref || '', scopeLabel: scopeOf(r).label
    }));
}

function listAuditLog(query, user) {
  query = query || {};
  // 审计日志刻意与其它列表不同：不带 limit/offset 时默认只回最近 100 条
  // (audit_log 只增不减，无参全量会随时间越滚越大)。只带 limit 的老调用
  // (如 /audit?limit=200) 按「第一页」处理，行为不变、只是多了 total/hasMore。
  if (query.limit === undefined && query.offset === undefined) {
    query = Object.assign({}, query, { limit: 100, offset: 0 });
  }
  const page = pageOf(query);

  const where = [], args = [];
  // 审计日志按「同范围内的人」过滤：部门范围看本部门账号的操作，带教范围只看自己。
  // scope=all (管理员) 不过滤 —— 与旧版语义一致。
  const scopeIds = user ? scopedLearnerIds(user) : null;
  if (scopeIds !== null) {
    const sc = scopeOf(user);
    const allowed = [user.id];
    if (sc.kind === 'dept' && sc.dept) {
      db.prepare('SELECT id FROM users WHERE dept = ?').all(sc.dept).forEach((u) => allowed.push
    }
    where.push(`user_id IN (${allowed.map(() => '?').join(',')})`);
    args.push(...allowed);
```

```
  }

  // 旧实现是「先 LIMIT 再 JS 过滤」，带范围的账号会看到**少于 limit** 的行且翻不了页 —— 顺手修正
  let sql = 'SELECT * FROM audit_log';
  if (where.length) sql += ' WHERE ' + where.join(' AND ');
  sql += ' ORDER BY id DESC';

  let total = null;
  if (page) {
    const cntSql = 'SELECT COUNT(*) AS n FROM audit_log' + (where.length ? ' WHERE ' + where.joi
    total = db.prepare(cntSql).get(...args).n;
    sql += ' LIMIT ? OFFSET ?';
    args.push(page.limit, page.offset);
  }
  const items = db.prepare(sql).all(...args).map((r) => ({
    id: r.id, at: r.at, userId: r.user_id,
    action: r.action, target: r.target, detail: r.detail
  }));
  return Object.assign({ items }, pageMeta(total === null ? items.length : total, page));
}

/* ================================================================== *
 * 教练动作（P5）：辅导记录 + 对话质检
 *
 * 权限模型与场景/任务**刻意不同**：
 *   - 场景编辑、任务下发是「内容运营」动作 → 卡 can_edit
 *   - 写辅导记录、做质检是**带教主管的本职** → 不卡 can_edit（教练没有这个权限位）
 *   这里只卡一件事：**数据可见范围**（只能读/写自己看得到的学员）。
 * ================================================================== */
/* 场景草稿生成（P31）：从真实客户对话提炼场景，需真实模型 */
const { generateSceneDraft } = require('./lib/scene-draft');

/* 辅导与质检模块已下沉到 lib/coach.js（P23） */
const {
  noteRow, qcRow, listCoachNotes, saveCoachNote, replyCoachNote, deleteCoachNote,
  QC_STATE, listQc, setQc, clearQc,
  cleanupSessions, sweepStaleSessions, setManualScore
} = require('./lib/coach');

/* ================================================================== *
 * 待建场景池（P12）：内容供给闭环
 *   教练在质检里标「这类问题话术没覆盖」→ 进池 → 运营认领 → 起草新场景。
 *   为什么不做成「自动生成场景」：场景是训练资产，草稿必须人确认；
 *   系统只负责把「实战暴露的问题」结构化地带到运营面前。
 * ================================================================== */
function ideaRow(r) {
  return {
    id: r.id, title: r.title, problem: r.problem, quote: r.quote || '',
    recordId: r.record_id || '', learnerId: r.learner_id || '', sceneId: r.scene_id || '',
    status: r.status, by: r.by_name || '', at: r.at || '',
```

```
    claimedBy: r.claimed_by || '', claimedAt: r.claimed_at || ''
  };
}

function listIdeas(query) {
  const q = query || {};
  let sql = 'SELECT * FROM scene_ideas';
  const args = [];
  if (q.status && q.status !== 'all') { sql += ' WHERE status = ?'; args.push(String(q.status));
  sql += ' ORDER BY at DESC, id DESC';
  const items = db.prepare(sql).all(...args).map(ideaRow);
  return { items, total: items.length, open: db.prepare("SELECT COUNT(*) AS n FROM scene_ideas W
}

/** 教练把一条成绩暴露的问题标记为「待建场景」 */
function createIdea(payload, userId) {
  const user = requireUser(userId);
  const title = String((payload && payload.title) || '').trim();
  const problem = String((payload && payload.problem) || '').trim();
  if (!title || !problem) {
    throw Object.assign(new Error('请填写标题与问题描述'), { status: 400 });
  }
  const rid = String((payload && payload.recordId) || '');
  let rec = null;
  if (rid) {
    rec = db.prepare('SELECT * FROM records WHERE id = ?').get(rid);
    if (!rec) throw Object.assign(new Error('成绩不存在'), { status: 404 });
    assertRecordScope(user, rec);   // 只能为自己的可见范围提问题
  }
  const id = 'SI-' + nowStamp().replace(/[-: ]/g, '').slice(2, 14) + '-' + Math.random().toStrin
  db.prepare(`INSERT INTO scene_ideas
    (id,title,problem,quote,record_id,learner_id,scene_id,status,by_name,by_id,at)
    VALUES (?,?,?,?,?,?,?,'open',?,?,?)`).run(
    id, title.slice(0, 120), problem.slice(0, 600), String((payload && payload.quote) || '').sli
    rid, rec ? rec.learner_id : String((payload && payload.learnerId) || ''),
    rec ? rec.scene_id : String((payload && payload.sceneId) || ''),
    user.name, user.id, nowStamp()
  );
  log(user.id, 'scene-idea.create', id, `待建场景：${title.slice(0, 40)}`);
  return { idea: ideaRow(db.prepare('SELECT * FROM scene_ideas WHERE id = ?').get(id)) };
}

/** 运营认领 / 关闭（内容治理动作，与场景编辑同权限） */
function setIdeaStatus(id, status, userId) {
  const user = requireUser(userId);
  assertCanAssignTask(user);   // 与任务下发/场景编辑同一权限档
  const row = db.prepare('SELECT * FROM scene_ideas WHERE id = ?').get(String(id || ''));
  if (!row) throw Object.assign(new Error('待建场景不存在'), { status: 404 });
  const st = String(status || '');
  if (st !== 'claimed' && st !== 'closed' && st !== 'open') {
```

```
    throw Object.assign(new Error('状态须为 claimed / closed / open'), { status: 400 });
  }
  const claimed = st === 'claimed';
  db.prepare('UPDATE scene_ideas SET status = ?, claimed_by = ?, claimed_at = ? WHERE id = ?').r
    st, claimed ? user.name : (st === 'open' ? '' : row.claimed_by || ''), claimed ? nowStamp()
  );
  log(user.id, 'scene-idea.' + st, row.id, `待建场景状态 ${row.status} → ${st}`);
  return { idea: ideaRow(db.prepare('SELECT * FROM scene_ideas WHERE id = ?').get(row.id)) };
}

/** 删除待建场景（误标/测试数据清理；与认领同权限） */
function deleteIdea(id, userId) {
  const user = requireUser(userId);
  assertCanAssignTask(user);
  const row = db.prepare('SELECT * FROM scene_ideas WHERE id = ?').get(String(id || ''));
  if (!row) throw Object.assign(new Error('待建场景不存在'), { status: 404 });
  db.prepare('DELETE FROM scene_ideas WHERE id = ?').run(row.id);
  log(user.id, 'scene-idea.delete', row.id, `删除待建场景：${row.title.slice(0, 40)}`);
  return { id: row.id, deleted: true };
}

function stats() {
  const one = (sql) => { const r = db.prepare(sql).get(); return r ? Object.values(r)[0] : 0; };
  return {
    scenes: one('SELECT COUNT(*) AS n FROM scenes'),
    published: one("SELECT COUNT(*) AS n FROM scenes WHERE publish='published'"),
    learners: one('SELECT COUNT(*) AS n FROM learners'),
    tasks: one('SELECT COUNT(*) AS n FROM tasks'),
    records: one('SELECT COUNT(*) AS n FROM records'),
    recordsFromLLM: one("SELECT COUNT(*) AS n FROM records WHERE source LIKE '%llm%'"),
    sessions: one('SELECT COUNT(*) AS n FROM sessions'),
    goldSet: one('SELECT COUNT(*) AS n FROM gold_set'),
    auditLog: one('SELECT COUNT(*) AS n FROM audit_log')
  };
}

module.exports = {
  getUser, requireUser, assertCanEdit, assertCanReview,
  scopeOf, scopedLearnerIds, inScope,
  listScenes, listPublishedScenes, getScene, saveScene, deleteScene, validateScene,
  listVersions, rollback, canAssign,
  submitReview, approveScene, rejectScene, offlineScene,
  getTask, saveTask, setTaskStatus, deleteTask, taskProgressOf, TASK_STATUS,
  assertLearnerInScope, assertRecordScope, listCoachNotes, saveCoachNote, replyCoachNote,
  deleteCoachNote, listQc, setQc, clearQc, QC_STATE, generateSceneDraft,
  // turnRow 一并导出：自测要断言「返回的 turns / 落库的 evidence / 逐轮接口」
  // 三处字段集完全一致，必须能拿到同一个映射函数来比对（铁律 8）
  turnRow, turn, finish, listRecords, listTurns, learnerTurns, deleteRecord,
  listLearners, listTasks, listUsers, listAuditLog, stats, cleanupSessions, sweepStaleSessions,
  listIdeas, createIdea, setIdeaStatus, deleteIdea, importLearners, deleteLearner, duplicateScen
```

```
};


/* ======== 文件: server/lib/core.js ======== */
'use strict';
/**
 * 公共基础层 (P22 技术债拆分)
 *
 * 从 service.js 抽出的「用户 / 权限 / 数据可见范围」——
 * 这一层不依赖任何业务模块 (只依赖 db)，是任务、场景、记录、辅导的共同底座。
 *
 * 为什么先拆这一层: service.js 1563 行里，权限判断散落在各业务函数开头，
 * 改一条权限规则要通读全文。集中后规则只有一处，业务模块只管业务。
 */

const dbm = require('../db');
const { db } = dbm;

/* ---------------- 分页 ----------------
 * 铁律 7: items 之外的四个字段只在这一处定义 (页面/任务/审计共用同一形状)。 */
function pageOf(query) {
  const raw = query || {};
  if (raw.limit === undefined && raw.offset === undefined) return null;
  const limit = Number(raw.limit);
  const offset = Number(raw.offset === undefined ? 0 : raw.offset);
  if (!Number.isInteger(limit) || limit < 1 || limit > 500) {
    throw Object.assign(new Error('分页参数校验未通过: limit 须为 1~500 的整数'), { status: 400 });
  }
  if (!Number.isInteger(offset) || offset < 0) {
    throw Object.assign(new Error('分页参数校验未通过: offset 须为 >= 0 的整数'), { status: 400 });
  }
  return { limit, offset };
}

function pageMeta(total, page) {
  if (!page) return { total, limit: total, offset: 0, hasMore: false };
  return { total, limit: page.limit, offset: page.offset, hasMore: page.offset + page.limit < to
}

/* ---------------- 用户 ---------------- */
function getUser(userId) {
  if (!userId) return null;
  return db.prepare('SELECT * FROM users WHERE id = ?').get(userId) || null;
}

/**
 * 取操作人。
 * ⚠ 这里**不回落管理员**——身份由路由层 (server/auth.js) 解析后写入
 *    body.userId / query.userId 传入。解析不到就是 401，避免"漏传 userId
 *    即获得超管权限"的越权口子。
```

```
 */
function requireUser(userId) {
  const u = getUser(userId);
  if (!u) throw Object.assign(new Error('未登录或登录状态已失效'), { status: 401 });
  return u;
}

/* ---------------- 权限 ---------------- */
function assertCanEdit(user) {
  if (!user.can_edit && user.role !== 'admin') {
    throw Object.assign(new Error(`角色「${user.role}」无场景编辑权限`), { status: 403 });
  }
}

function assertCanReview(user, sceneId) {
  if (!user.can_review && user.role !== 'admin') {
    throw Object.assign(new Error(`角色「${user.role}」无审核权限`), { status: 403 });
  }
  // 审核角色分离：提交人不能审自己提交的内容
  const au = db.prepare('SELECT * FROM scene_audit WHERE scene_id = ?').get(sceneId);
  if (au && au.submit_by && au.submit_by === user.name) {
    throw Object.assign(new Error('审核角色分离：提交人不能审核自己提交的场景'), { status: 403 });
  }
}

/* ---------------- 数据可见范围 ----------------
 * users.data_scope: all 全量 / dept 仅本部门 / mentor 仅本人带教
 * users.scope_ref : scope=mentor 时匹配 learners.coach 的值
 * 统一出口 scopedLearnerIds(user)：返回 null 表示不限制
 */
function scopeOf(user) {
  const kind = (user && user.data_scope) || 'all';
  return {
    kind,
    dept: (user && user.dept) || '',
    ref: (user && user.scope_ref) || '',
    label: kind === 'all' ? '全部数据'
      : kind === 'dept' ? `本部数据 · ${(user && user.dept) || '未设部门'}`
        : kind === 'mentor' ? `本人带教 · ${(user && user.scope_ref) || '未绑定导师名'}`
          : '全部数据'
  };
}

// 学员集合变动不频繁，做 5 秒进程内缓存，避免每次查询都打库
const _scopeCache = new Map();
function scopedLearnerIds(user) {
  const sc = scopeOf(user);
  if (sc.kind === 'all') return null;
  const key = `${sc.kind}|${sc.dept}|${sc.ref}`;
  const hit = _scopeCache.get(key);
```

```
  if (hit && Date.now() - hit.at < 5000) return hit.ids;

  let ids = [];
  if (sc.kind === 'dept') {
    if (sc.dept) ids = db.prepare('SELECT id FROM learners WHERE dept = ?').all(sc.dept).map((r)
  } else if (sc.kind === 'mentor') {
    if (sc.ref) ids = db.prepare('SELECT id FROM learners WHERE coach = ?').all(sc.ref).map((r)
  } else {
    // 未知范围类型按最保守处理：什么都看不到（而不是误放行）
    ids = [];
  }
  _scopeCache.set(key, { at: Date.now(), ids });
  return ids;
}

function inScope(user, learnerId) {
  const ids = scopedLearnerIds(user);
  return ids === null || ids.indexOf(String(learnerId)) !== -1;
}

/* ---------------- 记录/学员级范围校验 ----------------
 * 两个都要用：做质检看的是记录、下发/导入看的是学员。集中在这里，
 * 免得每个业务模块自己写一遍 inScope 判断（写漏就是越权）。 */
function assertLearnerInScope(user, learnerId) {
  const lid = String(learnerId == null ? '' : learnerId);
  if (!lid) throw Object.assign(new Error('缺少学员 id'), { status: 400 });
  if (!db.prepare('SELECT 1 FROM learners WHERE id = ?').get(lid)) {
    throw Object.assign(new Error(`学员不存在：${lid}`), { status: 400 });
  }
  if (!inScope(user, lid)) {
    throw Object.assign(new Error('该学员不在你的数据可见范围内'), { status: 403 });
  }
}

function assertRecordScope(user, rec, what) {
  const lid = String((rec && rec.learner_id) == null ? '' : (rec && rec.learner_id));
  if (!lid) {
    // what 由调用方给出（做质检 / 查看逐轮对话），否则报错文案会把
    // 「看对话」说成「做质检」——错误信息误导排查方向比没信息更坏
    throw Object.assign(new Error(`该成绩未归属学员（练习未带 learnerId），无法${what || '做质检'}`)
  }
  if (!inScope(user, lid)) {
    throw Object.assign(new Error('该成绩不在你的数据可见范围内'), { status: 403 });
  }
}

module.exports = {
  getUser, requireUser, assertCanEdit, assertCanReview,
  scopeOf, scopedLearnerIds, inScope,
  pageOf, pageMeta, assertLearnerInScope, assertRecordScope
```

```
};


/* ======== 文件: server/lib/scenes.js ======== */
'use strict';
/**
 * 场景模块 (P22 技术债拆分)
 *
 * 从 service.js 抽出的「场景库 CRUD / 版本 / 审批 / 校验 / 审计」。
 * 这一块是原来最大的单块 (~300 行): 内容治理的闸门都在这儿——
 *   canAssign (只有已发布能下发)、提交审核、审批人不能审自己提交的、
 *   已发布被改自动回 review。
 *
 * 依赖: lib/core (权限) + db。不依赖任务/记录模块。
 */

const dbm = require('../db');
const { db, log, sceneRow, nowStamp, today, J, P } = dbm;
const {
  getUser, requireUser, assertCanEdit, assertCanReview,
  scopeOf, scopedLearnerIds, inScope, pageOf, pageMeta,
  assertLearnerInScope, assertRecordScope
} = require('./core');

function listScenes(query) {
  let rows = db.prepare('SELECT * FROM scenes ORDER BY id').all().map(sceneRow);
  if (query && query.publish) rows = rows.filter((s) => s.publish === query.publish);
  if (query && query.assignable === '1') rows = rows.filter((s) => s.publish === 'published');
  return rows.map((s) => Object.assign(s, {
    audit: auditOf(s.id),
    versions: countVersions(s.id)
  }));
}

/* ---------------------------------------------------------------- *
 * 学员端场景池 (P6): 练习通道可匿名读取，但**只给已发布**场景。
 *
 * 为什么需要它: 学员端原先只读页面加载时的 window.SCENES (静态 app-data.js)，
 * 所以运营在剧本编辑器里新建/改动的场景，学员**刷新也拿不到**——
 * 「下发 → 学员练」这条 P4 闭环对新场景是断的。
 *
 * 两个刻意的收口:
 *   1. 只返回 publish==='published'。下线会把 publish 置为 'offline'，
 *      所以草稿 / 待审 / 已下线都天然拿不到。
 *   2. 只返回学员真的要用的字段，**不带 audit / versions / owner / publish**——
 *      这是匿名接口，不该把审阅人、版本数、发布状态这些运营信息漏出去。
 * ---------------------------------------------------------------- */
function sceneForLearner(s) {
  return {
    id: s.id, code: s.code, name: s.name, subtitle: s.subtitle || '',
```

```
    avatarText: s.avatarText || (s.name || '?').slice(0, 1),
    color: s.color || '#5B8FF9',
    taskName: s.taskName || '', channel: s.channel || '',
    category: s.category || '', difficulty: s.difficulty || '',
    passLine: s.passLine || 0, duration: s.duration || 0,
    // 会话列表的展示字段（学员端沿用旧形状）
    last: s.last || '', time: s.time || '', unread: s.unread || 0,
    // 训练内容：没有这些的场景是不可训练的
    script: s.script || [], tips: s.tips || [],
    objectives: s.objectives || [], voicePool: s.voicePool || []
  };
}

function listPublishedScenes() {
  return db.prepare("SELECT * FROM scenes WHERE publish = 'published' ORDER BY id").all()
    .map(sceneRow).map(sceneForLearner);
}

function auditOf(sceneId) {
  if (sceneId == null || sceneId === '') {
    return { status: 'draft', submitBy: '', submitAt: '', reviewBy: '', reviewAt: '', rejectReas
  }
  const a = db.prepare('SELECT * FROM scene_audit WHERE scene_id = ?').get(String(sceneId));
  if (!a) return { status: 'draft', submitBy: '', submitAt: '', reviewBy: '', reviewAt: '', reje
  return {
    status: a.status,
    submitBy: a.submit_by || '', submitAt: a.submit_at || '',
    reviewBy: a.review_by || '', reviewAt: a.review_at || '',
    rejectReason: a.reject_reason || ''
  };
}

function countVersions(sceneId) {
  if (sceneId == null || sceneId === '') return 0;
  const r = db.prepare('SELECT COUNT(*) AS n FROM scene_versions WHERE scene_id = ?').get(String
  return r ? r.n : 0;
}

function getScene(id) {
  // ⚠ node:sqlite 不接受 undefined 作为绑定值（与 better-sqlite3 行为不同），
  // 新建场景时 input.id 为空会走到这里，必须显式判空。
  if (id === undefined || id === null || id === '') return null;
  const s = sceneRow(db.prepare('SELECT * FROM scenes WHERE id = ?').get(String(id)));
  if (!s) return null;
  s.audit = auditOf(id);
  s.versions = countVersions(id);
  return s;
}

function canAssign(sceneId) {
```

```
  if (sceneId == null || sceneId === '') return false;
  const s = db.prepare('SELECT publish FROM scenes WHERE id = ?').get(String(sceneId));
  return !!s && s.publish === 'published';
}

/** 校验场景（与前端编辑器同一套规则，服务端必须再验一遍） */
function validateScene(s) {
  const out = [];
  const push = (lv, msg) => out.push({ lv, msg });
  if (!s.name || !String(s.name).trim()) push('bad', '场景名称不能为空');
  if (!s.brief || String(s.brief).trim().length < 10) push('warn', '场景简介过短（建议 ≥ 10 字）')
  if (!(s.passLine >= 0 && s.passLine <= 100)) push('bad', '达标线需在 0–100 之间');
  if (!s.objectives || !s.objectives.length) push('bad', '至少需要 1 条训练目标');
  else {
    const empty = s.objectives.filter((o) => !o.text || !String(o.text).trim()).length;
    if (empty) push('bad', `有 ${empty} 条训练目标内容为空`);
  }
  if (!s.script || !s.script.length) push('bad', '至少需要 1 轮客户台词');
  else {
    const e = s.script.filter((x) => !x.t || !String(x.t).trim()).length;
    if (e) push('bad', `有 ${e} 轮客户台词为空`);
  }
  if (s.objectives && s.script && s.objectives.length !== s.script.length) {
    push('warn', `训练目标 ${s.objectives.length} 条 vs 剧本 ${s.script.length} 轮，两者不等时多出的
  }
  if (!s.voicePool || !s.voicePool.length) push('warn', '未配置语音脚本池');
  return out;
}

function nextSceneSeq() {
  const r = db.prepare("SELECT v FROM meta WHERE k = 'scene_seq'").get();
  let n = r ? parseInt(r.v, 10) : 4;
  for (;;) {
    const id = 'SC' + String(n).padStart(2, '0');
    if (!db.prepare('SELECT 1 FROM scenes WHERE id = ?').get(id)) {
      db.prepare('INSERT OR REPLACE INTO meta (k,v) VALUES (?,?)').run('scene_seq', String(n + 1
      return id;
    }
    n++;
  }
}

/**
 * 保存场景：写主表 + 归档版本 + 已发布场景自动回待审
 * @returns {object} 保存后的场景
 */
function saveScene(input, userId) {
  const user = requireUser(userId);
  assertCanEdit(user);

```

```
  const cur = getScene(input.id);
  const isNew = !cur;

  const v = validateScene(input);
  const bad = v.filter((x) => x.lv === 'bad');
  if (bad.length) {
    throw Object.assign(new Error('校验未通过：' + bad.map((b) => b.msg).join('；')), { status: 4
  }

  const at = nowStamp();
  const id = input.id || nextSceneSeq();
  const code = input.code || ('SC-' + id);

  let publish = input.publish || (cur ? cur.publish : 'draft');
  let note = '保存修改';
  if (isNew) { publish = 'draft'; note = '新建场景'; }
  else if (publish === 'published') { publish = 'review'; note = '变更已发布场景'; }

  const payload = {
    script: input.script || [], objectives: input.objectives || [],
    voicePool: input.voicePool || [], tips: input.tips || []
  };

  const rec = Object.assign({}, cur || {}, input, {
    id, code, publish, updatedAt: today(),
    owner: input.owner || (cur && cur.owner) || user.name
  });

  db.prepare(`INSERT INTO scenes
    (id,code,name,subtitle,avatar_text,color,task_name,channel,category,difficulty,
     pass_line,duration,status,publish,owner,updated_at,brief,payload)
    VALUES (?,?,?,?,?,?,?,?,?,?,?,?,?,?,?,?,?,?)
    ON CONFLICT(id) DO UPDATE SET
      code=excluded.code, name=excluded.name, subtitle=excluded.subtitle,
      avatar_text=excluded.avatar_text, color=excluded.color, task_name=excluded.task_name,
      channel=excluded.channel, category=excluded.category, difficulty=excluded.difficulty,
      pass_line=excluded.pass_line, duration=excluded.duration, status=excluded.status,
      publish=excluded.publish, owner=excluded.owner, updated_at=excluded.updated_at,
      brief=excluded.brief, payload=excluded.payload`)
    .run(rec.id, rec.code, rec.name, rec.subtitle || '', rec.avatarText || '', rec.color || '#5B
      rec.taskName || '', rec.channel || '', rec.category || '', rec.difficulty || '',
      rec.passLine || 0, rec.duration || 0, rec.status || 'active', publish,
      rec.owner, rec.updatedAt, rec.brief || '', J(Object.assign({}, rec, payload)));

  // 归档版本
  const n = countVersions(rec.id) + 1;
  db.prepare('INSERT INTO scene_versions (scene_id,v,at,by_user,note,snapshot) VALUES (?,?,?,?,?
    .run(rec.id, n, at, user.name, note, J(Object.assign({}, rec, payload)));

  // 审计记录
```

```
  const au = auditOf(rec.id);
  const nextStatus = publish === 'published' ? 'published' : (publish === 'review' ? 'review' :
  db.prepare(`INSERT INTO scene_audit (scene_id,status,submit_by,submit_at,review_by,review_at,r
    VALUES (?,?,?,?,?,?,?)
    ON CONFLICT(scene_id) DO UPDATE SET status=excluded.status,
      submit_by=excluded.submit_by, submit_at=excluded.submit_at,
      review_by=excluded.review_by, review_at=excluded.review_at, reject_reason=excluded.reject_
    .run(rec.id, nextStatus, au.submitBy || '', au.submitAt || '', au.reviewBy || '', au.reviewA

  log(user.id, isNew ? 'scene.create' : 'scene.save', rec.id, `${rec.name} → v${n} (${note}) `);
  return getScene(rec.id);
}

/**
 * 复制场景为草稿（P14）
 *
 * 运营最常做的事是「基于现有场景改一版」。重新建空场景再逐字段填太折磨，
 * 所以给一键复制：复制体不传 id/code（由 saveScene 生成），强制 draft 并走正常审核流程——
 * 复制品绝不能绕过「提交审核 → 审批发布」的闸门。
 */
function duplicateScene(id, userId) {
  const user = requireUser(userId);
  assertCanEdit(user);
  const src = getScene(id);
  if (!src) throw Object.assign(new Error('场景不存在'), { status: 404 });
  const clone = {
    name: String(src.name || '场景') + '（副本）',
    subtitle: src.subtitle || '',
    avatarText: src.avatarText || '',
    color: src.color || '#5B8FF9',
    taskName: src.taskName || '',
    channel: src.channel || '',
    category: src.category || '',
    difficulty: src.difficulty || '',
    passLine: src.passLine || 60,
    duration: src.duration || 0,
    status: 'active',
    brief: src.brief || '',
    script: src.script || [],
    objectives: src.objectives || [],
    voicePool: src.voicePool || [],
    tips: src.tips || []
  };
  const saved = saveScene(clone, userId);   // 不传 id → 新建 + 强制 draft
  log(user.id, 'scene.duplicate', saved.id, `复制场景：${src.name} → ${saved.name}`);
  return saved;
}

function deleteScene(id, userId) {
  const user = requireUser(userId);
```

```
  assertCanEdit(user);
  const s = getScene(id);
  if (!s) throw Object.assign(new Error('场景不存在'), { status: 404 });
  db.prepare('DELETE FROM scenes WHERE id = ?').run(id);
  db.prepare('DELETE FROM scene_audit WHERE scene_id = ?').run(id);
  log(user.id, 'scene.delete', id, s.name);
  return { ok: true };
}

function listVersions(sceneId) {
  return db.prepare('SELECT v,at,by_user AS byUser,note,snapshot FROM scene_versions WHERE scene
    .all(sceneId)
    .map((r) => ({ v: r.v, at: r.at, by: r.byUser, note: r.note, snapshot: P(r.snapshot, {}) }))
}

function rollback(sceneId, v, userId) {
  const user = requireUser(userId);
  assertCanEdit(user);
  const row = db.prepare('SELECT * FROM scene_versions WHERE scene_id = ? AND v = ?').get(sceneI
  if (!row) throw Object.assign(new Error('版本不存在'), { status: 404 });
  const snap = P(row.snapshot, {});
  snap.id = sceneId;
  snap.publish = 'draft';
  const saved = saveScene(snap, userId);
  // saveScene 已归档一次；补一条明确语义的版本
  const n = countVersions(sceneId) + 1;
  db.prepare('INSERT INTO scene_versions (scene_id,v,at,by_user,note,snapshot) VALUES (?,?,?,?,?
    .run(sceneId, n, nowStamp(), user.name, `回滚到 v${v}`, J(saved));
  db.prepare('UPDATE scene_audit SET status=?, submit_by=?, submit_at=?, reject_reason=? WHERE s
    .run('draft', '', '', '', sceneId);
  log(user.id, 'scene.rollback', sceneId, `回滚到 v${v}`);
  return getScene(sceneId);
}

/* ---- 审批流 ---- */
function submitReview(sceneId, userId) {
  const user = requireUser(userId);
  assertCanEdit(user);
  const s = getScene(sceneId);
  if (!s) throw Object.assign(new Error('场景不存在'), { status: 404 });
  const bad = validateScene(s).filter((x) => x.lv === 'bad');
  if (bad.length) throw Object.assign(new Error('存在阻断问题，无法提交：' + bad.map((b) => b.msg)

  db.prepare(`INSERT INTO scene_audit (scene_id,status,submit_by,submit_at,review_by,review_at,r
    VALUES (?,?,?,?,?,?,?)
    ON CONFLICT(scene_id) DO UPDATE SET status='review', submit_by=excluded.submit_by,
      submit_at=excluded.submit_at, reject_reason=''`)
    .run(sceneId, 'review', user.name, nowStamp(), '', '');
  db.prepare('UPDATE scenes SET publish = ? WHERE id = ?').run('review', sceneId);
  log(user.id, 'scene.submit', sceneId, s.name);
```

```
  return getScene(sceneId);
}

function approveScene(sceneId, userId) {
  const user = requireUser(userId);
  assertCanReview(user, sceneId);
  db.prepare(`UPDATE scene_audit SET status='published', review_by=?, review_at=?, reject_reason
    .run(user.name, nowStamp(), sceneId);
  db.prepare('UPDATE scenes SET publish = ? WHERE id = ?').run('published', sceneId);
  log(user.id, 'scene.approve', sceneId, '');
  return getScene(sceneId);
}

function rejectScene(sceneId, reason, userId) {
  const user = requireUser(userId);
  assertCanReview(user, sceneId);
  db.prepare(`UPDATE scene_audit SET status='draft', reject_reason=? WHERE scene_id=?`)
    .run((reason || '').trim() || '未填写原因', sceneId);
  db.prepare('UPDATE scenes SET publish = ? WHERE id = ?').run('draft', sceneId);
  log(user.id, 'scene.reject', sceneId, reason || '');
  return getScene(sceneId);
}

function offlineScene(sceneId, userId) {
  const user = requireUser(userId);
  assertCanReview(user, sceneId);
  db.prepare(`UPDATE scene_audit SET status='offline' WHERE scene_id=?`).run(sceneId);
  db.prepare('UPDATE scenes SET publish = ? WHERE id = ?').run('offline', sceneId);
  log(user.id, 'scene.offline', sceneId, '');
  return getScene(sceneId);
}

module.exports = {
  listScenes, sceneForLearner, listPublishedScenes, auditOf, countVersions, getScene,
  canAssign, validateScene, nextSceneSeq, saveScene, duplicateScene, deleteScene,
  listVersions, rollback, submitReview, approveScene, rejectScene, offlineScene
};


/* ======== 文件: server/lib/tasks.js ======== */
'use strict';
/**
 * 任务模块（P22 技术债拆分）
 *
 * 从 service.js 抽出的「下发 / 改期 / 改线 / 结束 / 删除」——
 * 指派关系落在 task_assignees，才能按学员反查「我的任务」。
 *
 * 依赖：lib/core（权限）+ db。校验场景存在时**延迟 require** scenes，
 * 避免与场景模块形成循环引用。
 */
```

```

const dbm = require('../db');
const { db, log, nowStamp, J, P } = dbm;
const {
  getUser, requireUser, assertCanEdit, assertCanReview,
  scopeOf, scopedLearnerIds, inScope, pageOf, pageMeta,
  assertLearnerInScope, assertRecordScope
} = require('./core');

/** 延迟取场景（任务校验要用「场景必须已发布」，但场景模块不该互相 require） */
function sceneOf(id) {
  // eslint-disable-next-line global-require
  return require('./scenes').getScene(id);
}

function assertCanAssignTask(user) {
  // 任务下发属于内容运营动作，与场景编辑同权限
  if (!user.can_edit && user.role !== 'admin') {
    throw Object.assign(new Error(`角色「${user.role}」无任务下发权限`), { status: 403 });
  }
}

function nextTaskCode() {
  const todayKey = nowStamp().slice(0, 10).replace(/-/g, '');
  const prefix = 'RW-' + todayKey + '-';
  const r = db.prepare('SELECT COUNT(*) AS n FROM tasks WHERE code LIKE ?').get(prefix + '%');
  return prefix + String((r ? r.n : 0) + 1).padStart(2, '0');
}

function assigneesOf(taskId) {
  if (taskId == null || taskId === '') return [];
  return db.prepare('SELECT learner_id FROM task_assignees WHERE task_id = ? ORDER BY learner_id
    .all(String(taskId)).map((r) => r.learner_id);
}

function taskRow(r) {
  if (!r) return null;
  const base = P(r.payload, {}) || {};
  return Object.assign({}, base, {
    id: r.id, code: r.code, title: r.title, sceneId: r.scene_id,
    passLine: r.pass_line, dueAt: r.due_at, status: r.status,
    createdAt: r.created_at || base.createdAt || '',
    startAt: r.start_at || base.startAt || '',
    createdBy: r.created_by || base.createdBy || '',
    requireAll: r.require_all == null ? base.requireAll !== false : !!r.require_all,
    note: r.note != null ? r.note : (base.note || '')
  });
}

function getTask(id) {
```

```
  if (id === undefined || id === null || id === '') return null;
  const t = taskRow(db.prepare('SELECT * FROM tasks WHERE id = ?').get(String(id)));
  if (!t) return null;
  t.assignees = assigneesOf(id);
  // 与 listTasks 保持同一形状：单条任务也要带 progress，
  // 否则「下发后立刻读回单个任务」拿不到进度（自测就是这样抓到的）
  t.progress = taskProgressOf(id, t.assignees, t.dueAt);
  return t;
}

/** 任务进度：以服务端记录为准（与前端 helpers.taskProgress 口径一致） */
function taskProgressOf(taskId, assignees, dueAt) {
  const list = assignees || assigneesOf(taskId);
  const rows = db.prepare('SELECT learner_id, score, passed, at FROM records WHERE task_id = ? O
  const latest = {};
  rows.forEach((r) => { latest[r.learner_id] = r; });
  const done = list.filter((lid) => latest[lid]);
  const passed = done.filter((lid) => latest[lid].passed);
  const scores = done.map((lid) => latest[lid].score || 0);
  const due = dueAt !== undefined ? dueAt : dueOf(taskId);
  const dueMs = due ? Date.parse(String(due).replace(/-/g, '/')) : 0;
  return {
    total: list.length,
    finishedCount: done.length,
    passedCount: passed.length,
    overdueCount: list.filter((lid) => !latest[lid] && dueMs && Date.now() > dueMs).length,
    avgScore: scores.length ? Math.round(scores.reduce((a, b) => a + b, 0) / scores.length) : 0,
    rate: list.length ? Math.round(done.length / list.length * 100) : 0,
    passRate: done.length ? Math.round(passed.length / done.length * 100) : 0,
    doneIds: done
  };
}

function dueOf(taskId) {
  const t = db.prepare('SELECT due_at FROM tasks WHERE id = ?').get(String(taskId));
  return t ? t.due_at : '';
}

/**
 * 查询任务
 * @param {object} query  支持 learnerId（学员端「我的任务」）、status、sceneId
 * @param {object} user   操作人；非 all 范围只返回指派给其可见学员的任务
 * @param {boolean} mineOnly  learnerId 传入时是否只返回指派给该学员的任务
 */
function listTasks(query, user, mineOnly) {
  query = query || {};
  const page = pageOf(query);
  let rows = db.prepare('SELECT * FROM tasks ORDER BY created_at DESC, id DESC').all().map(taskR

  if (query.status) rows = rows.filter((t) => t.status === query.status);
```

```
  if (query.sceneId) rows = rows.filter((t) => t.sceneId === query.sceneId);

  // 学员端：只看得到派给自己的、且已下发的任务
  if (mineOnly && query.learnerId) {
    const lid = String(query.learnerId);
    rows = rows.filter((t) => t.status !== 'draft' && assigneesOf(t.id).indexOf(lid) !== -1);
  }

  const scopeIds = user ? scopedLearnerIds(user) : null;
  const items = rows.map((t) => {
    let assignees = assigneesOf(t.id);
    if (scopeIds !== null) assignees = assignees.filter((x) => scopeIds.indexOf(x) !== -1);
    return Object.assign(t, {
      assignees,
      progress: taskProgressOf(t.id, assignees, t.dueAt),
      visibleAssignees: assignees.length,
      totalAssignees: assigneesOf(t.id).length
    });
  }).filter((t) => scopeIds === null || t.assignees.length > 0 || t.totalAssignees === 0);
  const sliced = page ? items.slice(page.offset, page.offset + page.limit) : items;
  return Object.assign({ items: sliced }, pageMeta(items.length, page));
}

/** 新建 / 更新任务（下发） */
function saveTask(input, userId) {
  const user = requireUser(userId);
  assertCanAssignTask(user);

  const cur = getTask(input.id);
  const isNew = !cur;
  const title = String(input.title || (cur && cur.title) || '').trim();
  const sceneId = String(input.sceneId || (cur && cur.sceneId) || '');
  const assignees = (input.assignees || (cur && cur.assignees) || []).map(String).filter(Boolean

  // 校验：与前端同一套规则，服务端必须再验一遍
  const bad = [];
  if (!title) bad.push('任务名称不能为空');
  if (!sceneId) bad.push('必须选择训练场景');
  const scene = sceneOf(sceneId);
  if (sceneId && !scene) bad.push('训练场景不存在');
  if (scene && !isNew && scene.publish !== 'published' && cur.sceneId !== sceneId) bad.push('只能
  if (isNew && scene && scene.publish !== 'published') bad.push('只能下发已发布场景（当前场景尚未审
  if (!assignees.length) bad.push('至少选择 1 名学员');
  const unknown = assignees.filter((lid) => !db.prepare('SELECT 1 FROM learners WHERE id = ?').g
  if (unknown.length) bad.push(`学员不存在：${unknown.join('、')}`);
  if (bad.length) throw Object.assign(new Error('校验未通过：' + bad.join('；')), { status: 400 }

  const passLine = Number(input.passLine != null ? input.passLine : (cur ? cur.passLine : scene.
  if (!(passLine >= 0 && passLine <= 100)) {
    throw Object.assign(new Error('校验未通过：达标线需在 0–100 之间'), { status: 400 });
```

```
  }

  const id = input.id || ('TN' + Date.now());
  const at = nowStamp();
  const rec = {
    id, code: input.code || (cur && cur.code) || nextTaskCode(),
    title, sceneId,
    passLine,
    dueAt: input.dueAt || (cur && cur.dueAt) || '',
    startAt: input.startAt || (cur && cur.startAt) || at,
    createdAt: (cur && cur.createdAt) || at,
    createdBy: (cur && cur.createdBy) || user.name,
    requireAll: input.requireAll != null ? !!input.requireAll : (cur ? cur.requireAll : true),
    status: input.status || (cur && cur.status) || 'running',
    note: input.note != null ? String(input.note) : ((cur && cur.note) || ''),
    assignees
  };

  db.prepare(`INSERT INTO tasks
    (id,code,title,scene_id,pass_line,due_at,status,created_at,payload,
     start_at,created_by,require_all,note,channel)
    VALUES (?,?,?,?,?,?,?,?,?,?,?,?,?,?)
    ON CONFLICT(id) DO UPDATE SET
      code=excluded.code, title=excluded.title, scene_id=excluded.scene_id,
      pass_line=excluded.pass_line, due_at=excluded.due_at, status=excluded.status,
      payload=excluded.payload, start_at=excluded.start_at, created_by=excluded.created_by,
      require_all=excluded.require_all, note=excluded.note, channel=excluded.channel`)
    .run(rec.id, rec.code, rec.title, rec.sceneId, rec.passLine, rec.dueAt, rec.status,
      rec.createdAt, J(rec), rec.startAt, rec.createdBy, rec.requireAll ? 1 : 0, rec.note, scene

  // 指派关系：整表替换（简单、可预期；任务指派规模很小）
  db.prepare('DELETE FROM task_assignees WHERE task_id = ?').run(rec.id);
  const insTA = db.prepare('INSERT OR IGNORE INTO task_assignees (task_id, learner_id) VALUES (?
  rec.assignees.forEach((lid) => insTA.run(rec.id, lid));

  log(user.id, isNew ? 'task.assign' : 'task.update', rec.id,
    `${rec.title} → ${rec.assignees.length} 人${isNew ? '（新下发）' : ''}`);
  return getTask(rec.id);
}

/** 改任务状态：running / finished / draft（draft = 收回未下发） */
const TASK_STATUS = ['running', 'finished', 'draft'];

function setTaskStatus(id, status, userId) {
  const user = requireUser(userId);
  assertCanAssignTask(user);
  const t = getTask(id);
  if (!t) throw Object.assign(new Error('任务不存在'), { status: 404 });
  if (TASK_STATUS.indexOf(status) === -1) {
/* ……（此处省略中间 9837 行）…… */
```

```
    var rvId = mrv.getAttribute('data-ms-revoke');
    if (!SYNC || !SYNC.patchRecordScore) { toast('服务端未连接，无法撤销'); return; }
    SYNC.patchRecordScore(rvId, null, null, true).then(function (res) {
      if (!res.ok) { toast('撤销失败：' + (res.error || '未知错误')); return; }
      var rec = T.records.filter(function (x) { return x.id === rvId; })[0];
      if (rec) rec.manualScore = null;
      closeDrawer(); renderQuality(); renderOverview();
      toast('已撤销修正，恢复 AI 原分 ' + res.data.aiScore + ' 分');
    });
    return;
  }

  // KPI 钻取（P14）：数字点进去看明细。放在所有具体动作之前，
  // 因为卡片内部的「高风险 N 人」也带 data-kpi-go，必须先命中它而不是外层卡片。
  if (e.target.closest('[data-act="weekly"]')) { openWeekly(); return; }
  if (e.target.closest('[data-weekly-close]')) { closeDrawer(); return; }
  var wc = e.target.closest('[data-weekly-copy]');
  if (wc) {
    var ta = document.getElementById('weeklyText');
    if (!ta) return;
    var txt = ta.value;
    if (navigator.clipboard && navigator.clipboard.writeText) {
      navigator.clipboard.writeText(txt).then(function () { toast('周报已复制到剪贴板'); })
        .catch(function () { ta.select(); toast('已选中全文，按 Ctrl+C 复制'); });
    } else { ta.select(); toast('已选中全文，按 Ctrl+C 复制'); }
    return;
  }
  var kg = e.target.closest('[data-kpi-go]');
  if (kg) {
    var spec = String(kg.getAttribute('data-kpi-go') || '').split(':');
    if (spec[0] === 'students') {
      location.hash = '#students';
      var sel = $('#stuRisk');
      if (sel) sel.value = spec[1] === 'red' ? 'red' : 'all';
      renderStudents();
    }
    e.stopPropagation();
    return;
  }

  // 多选列：让浏览器处理 checkbox，不要触发行级「打开辅导抽屉」
  if (e.target.closest('[data-noclick]')) return;
  var ex = e.target.closest('[data-act="export-csv"]');
  if (ex) { exportCsv(); return; }
  var br = e.target.closest('[data-act="bulk-remind"]');
  if (br) { bulkRemind(); return; }
  var bc = e.target.closest('[data-act="bulk-clear"]');
  if (bc) { clearPicks(); return; }

```

```
    var go = e.target.closest('[data-go-coach]');
    if (go) {
      // 行内按钮自行处理点击，不要让整行兜底逻辑重复触发
      if (go.tagName === 'TR' && e.target.closest('button[data-go-coach]')) {
        openLearnerDrawer(e.target.closest('button[data-go-coach]').getAttribute('data-go-coach'
        return;
      }
      if (go.tagName === 'TR' && e.target.closest('button')) return;
      openLearnerDrawer(go.getAttribute('data-go-coach'));
      return;
    }
  });

  window.addEventListener('hashchange', route);
  route();
  boot();   // 装载：本地缓存先渲染 → 探活 → 身份 → 水合 → 拉辅导/质检 → 重渲染

  window.__probe = function () {
    var ov = H.overview();
    var ids = learnerIds();
    return [
      'scope=' + scope,
      'mine=' + ids.length,
      'kpis=' + $$('#kpiRow .kpi').length,
      'todos=' + $$('#todoList .prio').length,
      'feed=' + $$('#feedList .ev').length,
      'focus=' + $$('#focusList .item').length,
      'gaps=' + $$('#gapList .gap').length,
      'riskSum=' + (ov.risk.red + ov.risk.yellow + ov.risk.green),
      'avg=' + ov.avgScore,
      'engine=' + (S ? 'on' : 'off'),
      // P5：身份与数据范围
      'srvOnline=' + srvOnline(),
      'loggedIn=' + !!(SYNC && SYNC.loggedIn),
      'authUser=' + (IDENT ? IDENT.id : ''),
      'staff=' + COACH,
      'authScope=' + SCOPE_KIND,
      'authScopeLabel=' + (IDENT ? (IDENT.scopeLabel || '') : ''),
      'learnerCount=' + T.learners.length,
      'recordCount=' + T.records.length,
      'noteCount=' + notes.length,
      'qcCount=' + Object.keys(qcState).length,
      // 探针读真实渲染状态而非 hidden 属性：display:flex 会覆盖 hidden（P11 真机踩坑）
      'loginBoxOpen=' + (getComputedStyle($('#loginBox')).display !== 'none')
    ].join('|');
  };
})();


/* ======== 文件：仿真培训原型-微信风格.html ======== */
```

```
1601
1602 /* ==================== 数据层 ====================
1603    维度 DIMS / 关键词 KW / 目标维度映射 OBJ_DIM_DEFAULT / 场景 SCENES
1604    统一切换到共享数据源 data/app-data.js：它把同名变量挂到 window 上，
1605    因此本文件下方所有 DIMS / KW / SCENES / OBJ_DIM_DEFAULT 的引用均无需改动。 */
1606
1607 /* ==================== 状态 ==================== */
1608 let cur = 0, step = 0, running = false, ended = false;
1609 let dimScore = [42,42,42,42,42];
1610 let objDone = [];
1611 /* 共享评分引擎的状态（按维度 id 存），与 dimScore 保持同步。引擎不可用时为空对象 */
1612 let dimState = { d1:42, d2:42, d3:42, d4:42, d5:42 };
1613 /* AI 通道的整场累计（巡检 P11 发现的口径漂移修复）：
1614  * 服务端 finish() 的综合分 = **各轮 round(score) 的平均**、各维终值 = **各轮维分平均**。
1615  * 前端原先每轮把服务端返回值「覆盖」进 dimState/显示值 → 报告页显示的是**最后一轮**，
1616  * 与落库（整场平均）差好几分，学员看到的分和教练端看到的记录对不上。
1617  * 现在前端与服务端同口径：累计各轮 → 取平均。 */
1618 let dimAccum = null, dimRounds = 0, scoreAccum = 0;
1619 let reviewed = [];     // 逐句点评
1620 let pendingWant = 0;
1621 let voiceMode = false;
1622 let callTimer = null, callSec = 0, callTurns = 0, inCall = false;
1623 let DEMO = false;
1624
1625 const $ = id => document.getElementById(id);
1626 const curScene = () => SCENES[cur];
1627
1628 /* ==================== 学员身份 ====================
1629    ?as=L01 可切换演示学员（默认 L01 张三，与 data/app-data.js 的 learners 对应）。
1630    只有「指派给我」的任务才会出现在「我的任务」里 —— 之前是列出全部任务，
1631    与"我的任务"这个标题不符。 */
1632 const ME = (function(){
1633   try{
1634     const q = new URLSearchParams(location.search);
1635     return q.get('as') || q.get('learner') || 'L01';
1636   }catch(e){ return 'L01'; }
1637 })();
1638 let curTaskId = '';      // 当前这场训练归属的任务（成绩带 taskId，后台任务进度才能推进）
1639 let curTaskPass = null;
1640 let curCardTask = null;   // 任务卡当前展示的任务（供「开始练习」直接开练）   // 任务可覆盖场景默认达标
1641 let serverTasks = null;  // 服务端任务快照；null = 离线，回退本地数据
1642
1643 function myName(){
1644   const l = window.TRAIN && TRAIN.learnerById && TRAIN.learnerById(ME);
1645   return l ? l.name : ME;
1646 }
1647
1648 /* 我的任务清单：服务端优先（管理员新下发的任务学员端才收得到），离线回退本地 */
1649 function taskFeed(){
1650   const local = (window.TRAIN && TRAIN.tasks) || [];
```

```
  const mine = list => (list || []).filter(t => (t.assignees || []).indexOf(ME) >= 0);
  return serverTasks ? mine(serverTasks) : mine(local);
}

/* 拉取服务端任务；失败静默（后端没起也照常离线练） */
function loadMyTasks(){
  if(!window.AIBOOT || !AIBOOT.tasks) return;
  AIBOOT.tasks(ME).then(list=>{
    if(list){
      /* 签名变化才打日志：轮询路径下每 60s 一条会把控制台刷成噪音 */
      const sig = list.map(t=>t.id + ':' + t.status).join('|');
      const changed = !serverTasks || serverTasks.map(t=>t.id + ':' + t.status).join('|') !== si
      serverTasks = list;
      renderTasks();
      initList();          // 任务到达/状态变化 → 会话列表的「任务在身」标记同步
      if(changed) console.log('[任务] 已从服务端同步 ' + list.length + ' 条我的任务');
    }
  }).catch(()=>{});
}

/* 拉取「已发布场景池」并热更新到场景库：
   运营在后台新发布/改过的场景，学员刷新后就能练到（否则场景库永远停在页面加载时的种子）。
   合并语义是「按 id 原地更新 + 新增追加，不重排不删除」——不重排是为了让 cur 下标仍指向同一场景。
   失败/离线一律静默，保持本地种子场景不变。 */
function loadScenes(){
  if(!window.AIBOOT || !AIBOOT.scenes) return;
  if(!window.TRAIN || !TRAIN.helpers || !TRAIN.helpers.mergeScenes) return;
  AIBOOT.scenes().then(list=>{
    if(!list || !list.length) return;             // null=服务端不可用；[]=服务端还没发布场景，都保持
    const r = TRAIN.helpers.mergeScenes(list);
    if(r.added || r.updated){
      initList();                                  // 会话列表重建
      renderTasks();                               // 任务的「场景已下线」态可能因此解除
      console.log('[场景] 已同步服务端场景池：新增 ' + r.added + ' 个 / 更新 ' + r.updated + ' 个');
    }
  }).catch(()=>{});
}

/* 场景 → 我在这个场景上的进行中任务（唯一查询口径，绑定与列表标记共用） */
function taskOfScene(sceneId){
  return taskFeed().filter(t => t.sceneId === sceneId && t.status === 'running')[0] || null;
}

/* 按场景自动绑定「我的进行中任务」——从会话列表直接开始也能正确归属成绩 */
function bindTaskForScene(sceneId){
  const hit = taskOfScene(sceneId);
  curTaskId = hit ? hit.id : '';
  curTaskPass = hit && hit.passLine ? hit.passLine : null;
  return hit || null;
}
```

```
1701
1702  /* 从任务卡片直接开始训练 */
1703  function startTask(taskId){
1704    const t = taskFeed().filter(x => x.id === taskId)[0];
1705    if(!t) return;
1706    const i = SCENES.map(s => s.id).indexOf(t.sceneId);
1707    if(i < 0){ toast2('该任务关联的场景不在当前场景库中（可能已被下线）'); return; }
1708    openChat(i, t);
1709  }
1710
1711  /* ==================== 初始化 ==================== */
1712  function initList(){
1713    $('convList').innerHTML = SCENES.map((s,i)=>{
1714      /* 场景上有我进行中的任务 → 「任务在身」实底绿标；否则是自主练习的「陪练」标 */
1715      const tk = taskOfScene(s.id);
1716      return `
1717      <div class="li" onclick="openChat(${i})">
1718        <div class="ava" style="background:${s.color}">${s.ava}</div>
1719        <div class="mid">
1720          <div class="r1"><span class="nm">${s.name}<span class="pill${tk?' on-task':''}">${tk?'任
1721          <div class="r2"><span class="ms">${s.last}</span>${s.unread?`<span class="badge">${s.unr
1722        </div>
1723      </div>`;
1724    }).join('');
1725  }
1726
1727  function initDims(){
1728    $('dimList').innerHTML = DIMS.map((d,i)=>`
1729      <div class="dim">
1730        <div class="top"><span>${d}</span><span id="dv${i}">${dimScore[i]}</span></div>
1731        <div class="bar"><i id="db${i}" style="width:${dimScore[i]}%"></i></div>
1732      </div>`).join('');
1733  }
1734
1735  /* 我的任务：只列「指派给我」的任务；服务端在线时以服务端为准（运营新下发的任务能收到） */
1736  function renderTasks(){
1737    const box = $('taskList');
1738    if(!box) return;
1739    if(!window.TRAIN){                    // 数据源缺失时降级，不影响其它功能
1740      console.warn('[TRAIN] 未找到 window.TRAIN，我的任务面板降级显示');
1741      box.innerHTML = '<div class="mini">任务数据未加载</div>';
1742      return;
1743    }
1744    $('taskOrg').textContent = TRAIN.meta.org;
1745    if($('taskWho')) $('taskWho').textContent = myName() + (serverTasks ? '（服务端同步）' : '（本地
1746
1747    const running = taskFeed().filter(t=>t.status === 'running');
1748    if(!running.length){
1749      box.innerHTML = '<div class="mini">暂无指派给你的进行中任务</div>';
1750      return;
```

```
  }
  box.innerHTML = running.map((t,i)=>{
    /* 服务端任务自带 progress（以服务端记录为准）；本地任务走共享口径 */
    const p = t.progress || (TRAIN.helpers && TRAIN.helpers.taskProgress(t.id)) ||
      {finishedCount:0, total:(t.assignees||[]).length};
    const scene = TRAIN.sceneById(t.sceneId);
    const left = leftDays(t.dueAt);
    const leftTxt = left < 0 ? '已逾期' : (left === 0 ? '今天到期' : '剩 ' + left + ' 天');
    const leftCol = left < 0 ? '#FA5151' : 'var(--acc-txt)';   // 逾期红，正常用页面强调绿
    const canStart = !!scene;
    const sep = i ? 'margin-top:10px;padding-top:4px;border-top:1px dashed var(--line)' : '';
    return `<div style="${sep}">
      <div class="kv"><span>任务名称</span><span>${esc(t.title)}</span></div>
      <div class="kv"><span>训练场景</span><span>${scene ? esc(scene.name) : esc(t.sceneId)}</spa
      <div class="kv"><span>达标线</span><span>${t.passLine} 分</span></div>
      <div class="kv"><span>截止时间</span><span>${String(t.dueAt).slice(5,16)}</span></div>
      <div class="kv"><span>剩余</span><span style="color:${leftCol}">${leftTxt}</span></div>
      <div class="kv"><span>完成进度</span><span>已完成 ${p.finishedCount} / ${p.total} 人</span><
      <div style="margin-top:8px;display:flex;gap:8px">
        <button class="s" data-start-task="${t.id}"${canStart?'':' disabled'} style="flex:1;heig
      </div>
    </div>`;
  }).join('');
}

/* 相对原型演示基准日 2026-10-05 计算剩余天数（负数为已逾期） */
function leftDays(dueAt){
  const due = new Date(String(dueAt).replace(/-/g,'/'));
  /* 基准日走 TRAIN.helpers.nowDate()（P26）——与教练台、管理端同一个"今天",
     否则同一任务学员端说"剩 5 天"、管理端说"已逾期 19 天"。 */
  const now = (window.TRAIN && TRAIN.helpers && TRAIN.helpers.nowDate)
    ? TRAIN.helpers.nowDate() : new Date();
  const today = new Date(now.getFullYear(), now.getMonth(), now.getDate());
  const dueDay = new Date(due.getFullYear(), due.getMonth(), due.getDate());
  return Math.round((dueDay - today) / 86400000);
}

function renderObjs(){
  const o = curScene().obj;
  $('objList').innerHTML = o.map((t,i)=>`
    <div class="obj ${objDone[i]?'done':''}"><div class="box"></div><div>${t}</div></div>`).join
}

/* ==================== 流程 ==================== */
/**
 * 任务卡渲染（P15）
 *
 * 原先这三项是写死的演示文案（"微信沟通"/"3 次"/"10-08 18:00"，且从不更新）。
 * 改为真实数据后出现新问题：**未进会话时整卡是"–"**，看起来像加载失败。
 * 所以统一成一个函数：进会话时用绑定任务，首页用「最近一条进行中任务」，
```

```
 * 两处都不留空。
 */
/**
 * 带教留言（P21）
 *
 * 教练在辅导抽屉里写的内容，此前学员完全看不到——学员练完只知道分数，不知道为什么。
 * 现在拉到并展示；**没有任何留言时整块隐藏**（不占位、不制造"我被忽视了"的暗示）。
 */
/**
 * 练习记录（P30）
 *
 * 之前学员只能看到「最近一次报告」+ 成长汇总，想回看"上周五那场为什么只有 50 分"没入口。
 * 这里列出自己的历史练习（服务端 /records 对匿名学员强制要求 learnerId，只返回本人）。
 * 没有记录就整卡隐藏——空列表只会让人以为功能坏了。
 */
async function renderHistory(){
  var card = document.getElementById('histCard');
  var box = document.getElementById('histList');
  if (!card || !box) return;
  var list = null;
  try {
    if (window.AIBOOT && AIBOOT.records) list = await AIBOOT.records(ME, 8);
  } catch (e) { list = null; }
  if (!list || !list.length) { card.style.display = 'none'; return; }
  box.innerHTML = list.map(function (r) {
    var sc = (window.TRAIN && TRAIN.sceneById) ? TRAIN.sceneById(r.sceneId) : null;
    var nm = sc ? sc.name : (r.sceneId || '练习');
    var md = String(r.at || '').slice(5, 10).replace('-', '/');
    return '<div class="hist-item" role="button" tabindex="0" ' +
        'onclick="openHistory(\'' + esc(r.id) + '\')" ' +
        'onkeydown="if(event.key===\'Enter\'||event.key===\' \'){event.preventDefault();openHist
        'aria-label="回看 ' + esc(nm) + ' 这一场">' +
      '<div class="hi-main"><div class="hi-t">' + esc(nm) + '</div>' +
      '<div class="hi-m">' + esc(md) + ' · ' + (r.passed ? '<span class="hi-ok">已达标</span>' :
      (r.turnCount ? ' · ' + r.turnCount + ' 轮' : '') + '</div></div>' +
      '<div class="hi-s' + (r.passed ? '' : ' no') + '">' + (Number(r.score) || 0) + '</div>' +
      '</div>';
  }).join('');
  card.style.display = 'block';
}

/**
 * 打开某一场的逐轮回看（P30）
 *
 * 数据走 /learner/records/:id/turns（服务端按记录归属校验，读不到别人的）。
 * 只读：不给"再练一次"之类的动作，避免和历史数据语义混淆。
 */
async function openHistory(recId){
  var scr = document.getElementById('histScreen');
  var body = document.getElementById('histBody');
```

```
  if (!scr || !body) return;
  body.innerHTML = '<div class="rcard" style="color:#8A8A8A">读取中…</div>';
  scr.classList.add('on');
  var d = null;
  try {
    if (window.AIBOOT && AIBOOT.learnerTurns) d = await AIBOOT.learnerTurns(recId, ME);
  } catch (e) { d = null; }
  if (!d) {
    body.innerHTML = '<div class="rcard">读不到这一场的记录（可能已被清理，或服务未连接）。</div>';
    return;
  }
  var sc = (window.TRAIN && TRAIN.sceneById) ? TRAIN.sceneById(d.sceneId) : null;
  document.getElementById('histTitle').textContent = '回看 · ' + (sc ? sc.name : '练习');

  var DIM_IDS = ['d1', 'd2', 'd3', 'd4', 'd5'];
  var dims = (d.dimScores || {});
  var dimRows = DIMS.map(function (name, i) {
    var v = dims[DIM_IDS[i]];
    if (typeof v !== 'number') return '';
    return '<div class="dimrow"><span>' + esc(name) + '</span>' +
      '<div class="bw" style="flex:1;max-width:120px"><div class="bar"><i class="' +
      (v >= 75 ? '' : (v >= 60 ? 'a' : 'r')) + '" style="width:' + Math.min(100, v) + '%"></i></
      '<b>' + v + '</b></div>';
  }).join('');

  var turns = (d.turns || []).map(function (t, i) {
    var lt = t.learnerText || t.learner_text || '';
    var cr = t.customerReply || t.customer_reply || '';
    return '<div class="hrev">' +
      '<div class="rq">客户：' + esc(cr) + '</div>' +
      '<div class="ra">你：' + esc(lt) + '</div>' +
      '<div class="rs">第 ' + (i + 1) + ' 轮 · 本轮 ' + (Number(t.score) || 0) + ' 分' +
      (t.objectiveDim ? ' · 考察 ' + esc(dimName(t.objectiveDim)) : '') + '</div>' +
      '</div>';
  }).join('');

  body.innerHTML =
    '<div class="rcard" style="text-align:center;padding:22px 16px">' +
      '<div style="font-size:44px;font-weight:600;line-height:1.1;color:' + (d.passed ? '#07803F
      '<div style="font-size:12.5px;color:#8A8A8A;margin-top:6px">' +
        esc(String(d.at || '').slice(0, 10)) + ' · ' + (d.passed ? '已达标' : '未达标') +
        ' · 共 ' + (d.turns || []).length + ' 轮</div>' +
    '</div>' +
    (dimRows ? '<div class="rcard"><h3>五维得分</h3>' + dimRows + '</div>' : '') +
    '<div class="rcard"><h3>逐轮对话</h3>' +
      (turns || '<div style="color:#8A8A8A;font-size:12.5px">这一场没有保存逐轮对话。</div>') +
    '</div>';
}

function closeHistory(){
```

```
  var scr = document.getElementById('histScreen');
  if (scr) scr.classList.remove('on');
}

/** 维度 id → 中文名（历史回看用；DIMS 是名字数组，顺序即 d1..d5） */
function dimName(id){
  var ids = ['d1', 'd2', 'd3', 'd4', 'd5'];
  var i = ids.indexOf(id);
  return i >= 0 ? DIMS[i] : id;
}

/**
 * 我的成长（P21）
 *
 * 阶段/目标/下一步全部走 helpers.growthOf（与教练台同一份定义）。
 * 学员此前只看到"这场 64 分"，看不到"我在第几周、这个月的目标是多少"。
 */
function renderGrowth(){
  var card = document.getElementById('growthCard');
  var box = document.getElementById('growthBody');
  if (!card || !box) return;
  var g = null;
  try { g = (window.TRAIN && TRAIN.helpers && TRAIN.helpers.growthOf) ? TRAIN.helpers.growthOf(M
  if (!g) { card.style.display = 'none'; return; }   // 入职日期缺失就整块隐藏，不编阶段
  var rows = g.focus.map(function (f) {
    var cls = f.gap >= 0 ? '' : (f.gap > -10 ? 'a' : 'r');
    return '<div class="gap"><div class="nm">' + esc(f.name) + '</div>' +
      '<div class="bw"><div class="bar"><i class="' + cls + '" style="width:' + Math.min(100, f.
      '<div class="vl">' + f.v + '</div></div>';
  }).join('');
  box.innerHTML =
    '<div class="gstage">' + esc(g.stage.name) +
      '  <span class="gtag">入职第 ' + (g.days + 1) + ' 天 · 目标 ' + g.stage.target + '</span></d
    '<div class="glead">' + esc(g.stage.lead || '') + '</div>' + rows +
    '<div class="gnext">' + esc(g.next) + '</div>';
  card.style.display = 'block';
}

async function renderCoachNotes(){
  var card = document.getElementById('cnoteCard');
  var box = document.getElementById('cnoteList');
  if (!card || !box) return;
  var list = null;
  try {
    if (window.AIBOOT && AIBOOT.coachNotes) list = await AIBOOT.coachNotes(ME);
  } catch (e) { list = null; }
  if (!list || !list.length) { card.style.display = 'none'; return; }

  /* 学员回复（P29）：回复也存成一条留言（type=学员回复，replyTo 指向父留言），
     这里按 replyTo 分组挂回父留言下面——不新建表、不新建接口，复用同一份数据。 */
```

```
  var byParent = {};
  list.forEach(function (n) {
    if (n.replyTo) (byParent[n.replyTo] = byParent[n.replyTo] || []).push(n);
  });
  var parents = list.filter(function (n) { return !n.replyTo; }).slice(0, 3);
  if (!parents.length) { card.style.display = 'none'; return; }

  box.innerHTML = parents.map(function (n) {
    var reps = (byParent[n.id] || []).sort(function (a, b) {
      return String(a.at || '') < String(b.at || '') ? -1 : 1;
    });
    var repHtml = reps.map(function (r) {
      return '<div class="cn-rep">' +
        '<div class="cn-t">' + esc(r.text) + '</div>' +
        '<div class="cn-m">我 · ' + esc(String(r.at || '').slice(5, 10)) + '</div>' +
        '</div>';
    }).join('');
    /* 已经回复过就不再显示「回复」—只允许一层（服务端同样只允许一层），
       否则会变成聊天室，而系统没有已读/未读模型。 */
    var canReply = !reps.length;
    var btn = canReply
      ? '<button class="cn-reply" onclick="toggleReply(' + "'" + esc(n.id) + "'" + ')">回复</butt
      : '';
    var editor = canReply
      ? ('<div class="cn-box" id="cnbox-' + esc(n.id) + '" style="display:none">' +
          '<textarea id="cntext-' + esc(n.id) + '" maxlength="500" ' +
            'placeholder="说说你试过之后的情况，主管能看到"></textarea>' +
          '<div class="cn-acts">' +
            '<button onclick="toggleReply(' + "'" + esc(n.id) + "'" + ')">取消</button>' +
            '<button class="pri" id="cnsend-' + esc(n.id) + '" onclick="sendReply(' + "'" + esc(
          '</div>' +
        '</div>')
      : '';
    return '<div class="cnote" id="cnote-' + esc(n.id) + '">' +
      '<div class="cn-t">' + esc(n.text) + '</div>' +
      '<div class="cn-m">' + esc(n.by || '带教主管') + ' · ' + esc(String(n.at || '').slice(0, 10
      repHtml + editor +
      '</div>';
  }).join('');
  card.style.display = 'block';
}

/** 展开/收起某条留言的回复输入框 */
function toggleReply(noteId){
  var box = document.getElementById('cnbox-' + noteId);
  if (!box) return;
  var open = box.style.display !== 'none';
  box.style.display = open ? 'none' : 'block';
  if (!open) {
    var ta = document.getElementById('cntext-' + noteId);
```

```
    if (ta) { try { ta.focus(); } catch (e) {} }
  }
}

/**
 * 发送学员回复（P29）
 *
 * 归属与层级由服务端校验（只能回自己的留言、只允许一层）。
 * 但失败必须给明确提示，否则学员点了发送没反应会以为系统坏了。
 */
async function sendReply(noteId){
  var ta = document.getElementById('cntext-' + noteId);
  var btn = document.getElementById('cnsend-' + noteId);
  if (!ta) return;
  var text = (ta.value || '').trim();
  if (!text) { toast2('先写点什么再发送'); try { ta.focus(); } catch (e) {} return; }
  if (!window.AIBOOT || !AIBOOT.replyCoachNote) { toast2('离线模式下不能回复，连上服务后再试'); ret
  if (btn) { btn.disabled = true; btn.textContent = '发送中…'; }
  var ok = null;
  try { ok = await AIBOOT.replyCoachNote(ME, noteId, text); } catch (e) { ok = null; }
  if (btn) { btn.disabled = false; btn.textContent = '发送'; }
  if (!ok) { toast2('发送失败，稍后再试'); return; }
  toast2('已回复，带教主管能看到');
  try { await renderCoachNotes(); } catch (e) {}
}

function renderTaskCard(bound){
  var fb = bound || null;
  if(!fb && typeof taskFeed === 'function'){
    var running = taskFeed().filter(function(t){ return t.status === 'running'; });
    fb = running[0] || null;
  }
  /* 注意：curScene 是脚本作用域内的声明，不挂在 window 上——
     用 window.curScene 判断会恒为 undefined，导致场景永远显示"自主选择"。 */
  var scene = null;
  if (fb) {
    scene = (window.TRAIN && TRAIN.sceneById) ? TRAIN.sceneById(fb.sceneId) : null;
  } else {
    try { scene = curScene(); } catch (e) { scene = null; }
  }
  var nameEl = document.getElementById('tName');
  if(!nameEl) return;
  nameEl.textContent = fb ? fb.title : (scene ? scene.task : '暂无任务（自主练习）');
  // 无任务时不要显示破折号：学员还没选场景，写明"自主练习，进入会话后按场景算"
  document.getElementById('tScene').textContent = scene ? scene.name : '自主选择';
  var passLine = (fb && fb.passLine) || (scene && scene.pass);
  document.getElementById('tPass').textContent = passLine ? (passLine + ' 分') : '按场景';
  var doneN = 0;
  try{
    if(window.TRAIN && TRAIN.records){
```

```
      doneN = TRAIN.records.filter(function(r){
        if(r.learnerId !== ME) return false;
        return fb ? r.taskId === fb.id : !r.taskId;
      }).length;
    }
  }catch(e){ doneN = 0; }
  document.getElementById('tRound').textContent = doneN + ' 次';
  document.getElementById('tDeadline').textContent =
    (fb && fb.dueAt) ? String(fb.dueAt).slice(0, 10) : '不限时';
  // 有任务且能找到对应场景时，才给「开始练习」—否则按钮是死路
  curCardTask = (fb && scene) ? fb : null;
  var startBtn = document.getElementById('taskStartBtn');
  if (startBtn) {
    if (curCardTask) {
      startBtn.style.display = 'block';
      startBtn.textContent = '开始练习';
      startBtn.title = '进入「' + scene.name + '」练习这个任务';
    } else {
      startBtn.style.display = 'none';
    }
  }
}

/** 从任务卡直接开练（P20）：省掉"自己去列表里找对应会话"这一步 */
function startCurrentTask(){
  if (!curCardTask) return;
  var idx = SCENES.findIndex(function (s) { return s.id === curCardTask.sceneId; });
  if (idx < 0) { toast2('该任务关联的场景已下线，请联系带教主管'); return; }
  openChat(idx, curCardTask);
}

function openChat(i, task){
  cur = i; step = 0; running = true; ended = false;
  dimScore = [42,42,42,42,42];
  dimAccum = null; dimRounds = 0; scoreAccum = 0;   // AI 通道整场累计（P11 口径对齐）
  // objDone 必须与目标清单等长：场景轮次(6)可能多于目标数(5)，
  // 若用轮次下标直接写入会撑出一个"不存在的目标"。
  objDone = (curScene().obj || []).map(function(){ return false; });
  reviewed = []; callTurns = 0;
  ttsStop();   // 换会话/重开时停掉上一场残留朗读
  dimState = { d1:42, d2:42, d3:42, d4:42, d5:42 };
  /* 任务归属：显式指定 → 用它；否则按场景自动绑定我进行中的任务 */
  const bound = task || bindTaskForScene(curScene().id);
  curTaskId = bound ? bound.id : '';
  curTaskPass = bound && bound.passLine ? bound.passLine : null;
  initDims(); renderObjs(); closeSheet(); $('recov').classList.remove('on');
  $('scrList').style.display='none';
  $('reportScreen').classList.remove('on');
  $('scrChat').style.display='flex';
  $('chatTitle').innerHTML = curScene().name + '<small>' + curScene().sub + '</small>';
```

```
  $('backNum').textContent = '1';
  renderTaskCard(bound);
  $('chatBody').innerHTML = `<div class="time-div">今天 ${nowTime()}</div>`;
  $('tipBox').innerHTML = curScene().tips[0];
  $('msgInput').value=''; onInput();
  if(!DEMO) setTimeout(()=>clientSay(0), 700);
}

/**
 * 报告页的「下一个任务」入口（P21）
 *
 * 练完一个任务后，此前只有「返回任务列表 / 再练一次」—学员得自己回去找下一个。
 * 如果还有别的进行中任务，直接给一个按钮过去。
 */
function renderNextTaskBtn(){
  var foot = document.querySelector('.rep-foot');
  if (!foot) return;
  var old = foot.querySelector('[data-next-task]');
  if (old) old.remove();
  var feed = [];
  try { feed = (typeof taskFeed === 'function') ? taskFeed() : []; } catch (e) { feed = []; }
  var next = feed.filter(function (t) { return t.status === 'running' && t.id !== curTaskId; })[
  if (!next) return;
  var btn = document.createElement('button');
  btn.className = 'p';
  btn.setAttribute('data-next-task', '1');
  btn.textContent = '练下一个任务';
  btn.title = next.title;
  btn.onclick = function () { startTaskById(next.id); };
  foot.appendChild(btn);
}

/** 按任务 id 直接开练（供报告页「下一个任务」用） */
function startTaskById(tid){
  var feed = [];
  try { feed = (typeof taskFeed === 'function') ? taskFeed() : []; } catch (e) { feed = []; }
  var t = feed.filter(function (x) { return x.id === tid; })[0];
  if (!t) { toast2('这个任务已经不在了'); return; }
  var idx = SCENES.findIndex(function (sc) { return sc.id === t.sceneId; });
  if (idx < 0) { toast2('该任务关联的场景已下线，请联系带教主管'); return; }
  openChat(idx, t);
}

function backToList(){
  running = false; ended = false; inCall = false;
  clearInterval(callTimer);
  $('callScreen').classList.remove('on');
  $('reportScreen').classList.remove('on');
  $('scrChat').style.display='none';
  $('scrList').style.display='flex';
```

```
}

function resetAll(){ initList(); backToList(); }

function restart(){
  $('reportScreen').classList.remove('on');
  openChat(cur);
}

/* ---------- 客户说话 ---------- */
function clientSay(idx){
  const sc = curScene();
  const d = sc.script[idx];
  if(!d) return;
  // 服务端 NPC 生成的回复优先；消费一次即清空，避免串场
  const useAI = !!(aiCustomerText && aiCustomerText.trim());
  const text = useAI ? aiCustomerText.trim() : d.t;
  const voice = useAI ? (aiVoiceSec || d.v) : d.v;
  aiCustomerText = null; aiVoiceSec = null;
  lastCustomerText = text;
  showTyping(()=>{
    addMsg('in', text, {voice: voiceMode ? voice : 0, ai: useAI});
    const tip = sc.tips[idx] || '';
    $('tipBox').innerHTML = useAI
      ? tip + '<span class="ai-src">AI 实时生成</span>'
      : tip + '<span class="ai-src off">脚本兜底</span>';
    pendingWant = idx;
  });
}

function showTyping(cb){
  const row = document.createElement('div');
  row.className = 'row in';
  row.id = 'typingRow';
  row.innerHTML = `<div class="ava" style="background:${curScene().color}">${curScene().ava}</di
    <div class="bub" style="padding:0"><div class="typing"><i></i><i></i><i></i></div></div>`;
  $('chatBody').appendChild(row); scrollBottom();
  setTimeout(()=>{ row.remove(); cb(); }, 850);
}

/* ---------- 消息渲染 ---------- */
function addMsg(side, text, opt={}){
  const row = document.createElement('div');
  row.className = 'row ' + side;
  const avaTxt = side==='in' ? curScene().ava : '我';
  const avaColor = side==='in' ? curScene().color : '#07C160';
  const ava = `<div class="ava" style="background:${avaColor}">${avaTxt}</div>`;
  let bub;
  if(opt.voice){
    const w = Math.min(200, 78 + opt.voice*7.5);
```

```
    const icon = `<svg width="16" height="16" viewBox="0 0 20 20" fill="none" stroke="currentCol
    bub = `<div class="bub voice" style="width:${w}px">${icon}<span class="dur">${opt.voice}''</
  } else {
    bub = `<div class="bub">${esc(text)}</div>`;
  }
  row.innerHTML = side==='in' ? ava+bub : bub+ava;
  if(opt.ai){
    const tag = document.createElement('div');
    tag.className = 'ai-tag';
    tag.textContent = '⚡ AI 实时回复';
    row.querySelector('.bub').appendChild(tag);
  }
  $('chatBody').appendChild(row); scrollBottom();
  if(side==='in') ttsSay(text);   // 客户说话 → 语音模式下朗读（P11 优化，TTS）
}

/* ====== 客户语音播报（P11 优化）：浏览器原生 Web Speech API，零依赖 ======
 * 只在「语音模式」下自动朗读客户消息（文字模式静默，避免打扰）。
 * 降级：浏览器不支持 / 系统无中文语音包 → 静默跳过，练习流程不受影响。
 * 开关：手机屏顶部扬声器按钮（toggleSpeaker）。 */
let ttsOn = true;
function ttsSupported(){ return typeof window !== 'undefined' && 'speechSynthesis' in window &&
function ttsSay(text){
  if(!ttsOn || !voiceMode || !text) return;
  if(!ttsSupported()) return;
  try{
    window.speechSynthesis.cancel();
    const clean = String(text).replace(/<[^>]+>/g, '').replace(/⚡ AI 实时回复/g, '').trim().slic
    if(!clean) return;
    const u = new SpeechSynthesisUtterance(clean);
    u.lang = 'zh-CN';
    u.rate = 1.02;
    u.pitch = 1;
    const voices = window.speechSynthesis.getVoices() || [];
    const zh = voices.filter(v=>/^zh/i.test(v.lang))[0]
      || voices.filter(v=>/Chinese|中文|Huihui|Xiaoxiao|Yaoyao/i.test(v.name))[0];
    if(zh) u.voice = zh;
    window.speechSynthesis.speak(u);
  }catch(e){ /* 播报失败绝不影响练习 */ }
}
/* 停止播报（挂机/结束/切走时调用，避免残留声音） */
function ttsStop(){ try{ if(ttsSupported()) window.speechSynthesis.cancel(); }catch(e){} }

function scrollBottom(){ const b=$('chatBody'); b.scrollTop = b.scrollHeight; }
const esc = s => String(s).replace(/[&<>"]/g,c=>({'&':'&amp;','<':'&lt;','>':'&gt;','"':'&quot;'

/* ---------- 学员发送 ---------- */
function onInput(){
  const has = $('msgInput').value.trim().length > 0;
  $('btnSend').classList.toggle('on', has);
```

```
  $('btnPlus').style.display = has ? 'none' : 'flex';
}

function sendText(){
  const v = $('msgInput').value.trim();
  if(!v || !running || ended || judging){ return; }
  $('msgInput').value=''; onInput();
  addMsg('out', v);
  judge(v);
}

/* AI 通道状态：'unknown' 未探测 / 'off' 离线走本地引擎 / 'on' 走服务端 AI */
let aiMode = 'unknown';
/* 评分通道细分（P26）：'llm' 真实模型 / 'mock' 本地模拟 / 'keyword' 服务端关键词 / 'unknown'
   只看 aiMode 会把 mock 说成"AI 评分"，评委问"接的哪个模型"就答不上。 */
let aiChannel = 'unknown';
let aiModelName = '';
let aiSessionId = null;      // 与后端会话一一对应，用于多轮上下文与落库
let aiLastTurn = null;       // 最近一轮服务端返回（用于报告页展示模型点评/置信度）
let aiFeedback = {};         // 轮次 -> 模型点评
let aiCustomerText = null;   // 服务端 NPC 生成的下一条客户回复（消费一次即清空）
let aiVoiceSec = null;       // 服务端 NPC 给出的语音时长（秒），voice 通道用
let aiRecordId = null;       // 本场落库后的 Record id
let lastCustomerText = '';   // 客户最近一句（用于报告页回显真实对话）
let judging = false;         // 正在等待服务端返回，避免重复提交
let aiFailStreak = 0;        // 服务端连续失败计数，达阈值自动降级本场

/* 探测后端；不可用则整场练习走本地引擎，体验不中断 */
if(window.AIBOOT){
  AIBOOT.probe().then(ok=>{
    aiMode = ok ? 'on' : 'off';
    if(ok){
      /* 判据与后端一致：baseUrl/model 含本地地址或 mock 字样 → 模拟通道 */
      const info = AIBOOT.info || {};
      const llm = info.llm || {};
      const isMock = /localhost|127\.0\.0\.1|0\.0\.0\.0|mock/i
        .test(String(llm.baseUrl || '') + ' ' + String(llm.model || ''));
      aiChannel = llm.configured ? (isMock ? 'mock' : 'llm') : 'keyword';
      aiModelName = llm.model || '';
      aiSessionId = AIBOOT.newSessionId();
      console.log('[AI] 已连接服务端：' + AIBOOT.base + '，本场会话 ' + aiSessionId);
    } else {
      console.log('[AI] 服务端不可用，使用本地关键词引擎离线评分');
    }
  }).catch(()=>{ aiMode='off'; });
}

/* 本地引擎评分（原逻辑，抽成函数供降级复用） */
function judgeLocal(text){
  const hits = [];
```

```
  let ok;
  const want = pendingWant;
  const map = curScene().objDim || OBJ_DIM_DEFAULT;
  const wd = map[Math.min(want, map.length-1)];

  if(window.SCORER){
    const r = SCORER.scoreTurn(text, {
      state: dimState, kw: TRAIN.kw, dims: TRAIN.dims,
      objectiveDim: TRAIN.dims[wd] ? TRAIN.dims[wd].id : null
    });
    TRAIN.dims.forEach((d,i)=>{ dimScore[i] = dimState[d.id]; });
    r.hits.forEach(id=>{ hits.push(TRAIN.helpers.dimIndex(id)); });
    ok = r.objectiveMet;
  } else {
    DIMS.forEach((d,i)=>{
      if(KW[d].some(k=>text.includes(k))){ dimScore[i] = Math.min(97, dimScore[i]+13); hits.push
    });
    ok = hits.includes(wd) || hits.length >= 2;
    if(ok && hits.includes(wd)) dimScore[wd] = Math.min(97, dimScore[wd]+6);
  }
  return { ok, hits };
}

async function judge(text){
  // 评分优先走共享引擎（data/scoring-engine.js）；若后端可用，则走服务端 AI
  //  （LLM 扮演客户 NPC + Rubric 评分官），失败自动回落本地，保证练习不中断。
  if(judging) return;
  judging = true;
  try{
    await judgeInner(text);
  } finally {
    judging = false;
  }
}

async function judgeInner(text){
  let hits = [];
  let ok;
  const want = pendingWant;

  if(aiMode === 'on'){
    try{
      const r = await AIBOOT.turn({
        sessionId: aiSessionId,
        taskId: curTaskId || '',
        learnerId: ME,
        sceneId: curScene().id,
        channel: voiceMode ? 'voice' : 'text',
        learnerText: text,
        /* ⚠ 必须传「学员实际看到的那句」（P28）
```

```
2351          * 客户回复是模型生成的（npcReply），存在 lastCustomerText 里；
2352          * 原来这里传的是剧本原文 script[want].t —— 从第二轮起，
2353          * 模型评分依据的"客户发言"和学员屏幕上显示的就**不是同一句**，
2354          * 会出现"客户明明说了 X，为什么说我没回应"的错判。
2355          * 优先用实际显示的，没有才回落剧本。 */
2356         customerText: lastCustomerText || (curScene().script[want] ? curScene().script[want].t :
2357       });
2358       if(r.ok){
2359         const d = r.data;
2360         aiFailStreak = 0;
2361         // 整场累计（与服务端 finish 同口径）：各轮维分/轮分累加后取平均，
2362         // 而不是「每轮覆盖」—否则报告页显示最后一轮、与落库分对不上（P11 巡检）
2363         if(!dimAccum) dimAccum = { d1:0, d2:0, d3:0, d4:0, d5:0 };
2364         TRAIN.dims.forEach(dim=>{ if(typeof d.dimScores[dim.id]==='number') dimAccum[dim.id] +=
2365         dimRounds++;
2366         if(typeof d.score==='number') scoreAccum += d.score;
2367         TRAIN.dims.forEach(dim=>{ dimState[dim.id] = Math.round(dimAccum[dim.id] / dimRounds); }
2368         TRAIN.dims.forEach((dim,i)=>{ dimScore[i] = dimState[dim.id]; });
2369         ok = !!(d.objectives && d.objectives.some(o=>o.achieved && TRAIN.dims[wdIndexOf(o.dim)]
2370         // 目标达成以服务端判定为准：本轮目标维度是否达成
2371         const myDim = (curScene().objDim||OBJ_DIM_DEFAULT)[Math.min(want,(curScene().objDim||OBJ
2372         const myDimId = TRAIN.dims[myDim] ? TRAIN.dims[myDim].id : null;
2373         const mo = (d.objectives||[]).filter(o=>o.dim===myDimId)[0];
2374         ok = mo ? !!mo.achieved : ok;
2375         aiLastTurn = d;
2376         if(d.feedback) aiFeedback[want] = d.feedback;
2377         if(d.customerReply && d.customerReply.text){
2378           aiCustomerText = d.customerReply.text;
2379           aiVoiceSec = d.customerReply.voiceSec || null;
2380         }
2381       } else {
2382         // 服务端连续失败达阈值 → 本场自动切回本地引擎，避免每轮都白等超时
2383         if(++aiFailStreak >= 2){ aiMode = 'off'; console.warn('[AI] 服务端连续失败，本场切换为本地引
2384         console.warn('[AI] 本轮服务端失败，回落本地引擎：' + r.error);
2385         const l = judgeLocal(text); ok = l.ok; hits = l.hits;
2386       }
2387     } catch(e){
2388       if(++aiFailStreak >= 2){ aiMode = 'off'; console.warn('[AI] 服务端异常，本场切换为本地引擎');
2389       console.warn('[AI] 异常，回落本地引擎：' + e.message);
2390       const l = judgeLocal(text); ok = l.ok; hits = l.hits;
2391     }
2392   } else {
2393     const l = judgeLocal(text); ok = l.ok; hits = l.hits;
2394   }
2395
2396   // 目标下标要钳到目标清单范围内（与 judgeLocal 的 map[Math.min(want, len-1)] 保持一致）
2397   const objIdx = Math.min(want, Math.max(0, (curScene().obj || []).length - 1));
2398   if(ok && !objDone[objIdx]) objDone[objIdx] = true;
2399   /* 点评要能解释"为什么没命中"（P21）：把本轮考察点与对应维度一起存下来，
2400      否则学员只看到"未命中考察点"，不知道考察的是什么。 */
```

```
  const _objRef = (curScene().obj || [])[objIdx] || null;
  reviewed.push({ q: lastCustomerText || (curScene().script[want] ? curScene().script[want].t :
                  a: text, ok, feedback: aiFeedback[want] || '',
                  objText: _objRef ? _objRef.text : '', objDim: _objRef ? _objRef.dim : '' });
  initDims(); renderObjs();

  // 下一步：等待本轮的 AI 客户回复已就绪（aiCustomerText 由 await 完成后写入），再让客户说话
  step++;
  const total = curScene().script.length;
  if(step >= total){
    // 最后一轮判分后要等 900ms 才出报告，这段窗口必须立刻停止接单，
    // 否则手快的学员能再发一条（多算一轮、objDone 被撑长、分数被改写）。
    running = false;
    $('btnSend').classList.remove('on');
    setTimeout(finish, 900);
  } else {
    setTimeout(()=>clientSay(step), 520);
  }
}

/* 维度名 -> 序号（服务端返回 o.dim 是 id，需转成 0..4） */
function wdIndexOf(dimId){
  const i = TRAIN.dims.findIndex(d=>d.id===dimId);
  return i<0?0:i;
}

/* ---------- 语音 ---------- */
function toggleVoiceMode(){
  voiceMode = !voiceMode;
  $('iboxWrap').style.display = voiceMode ? 'none' : 'flex';
  $('holdBtn').classList.toggle('on', voiceMode);
  if(voiceMode) $('btnVoice').style.color = '#07C160';
  else $('btnVoice').style.color = '';
}

/* ====== 真语音识别（P11）：本地 FunASR · SenseVoiceSmall ======
 * 三重能力门：浏览器有麦克风 API + AIBOOT 在线 + 后端报告 ASR 在线。
 * 任何一环不满足（file:// 打开、无麦克风、ASR 服务未起）→ 回退原「话术选择」体验，
 * 原路径一个字不改 —— 已验收交付物的降级语义零破坏。 */
let rec_ = null, recT0 = 0;
function voiceCapable(){
  return !!(navigator.mediaDevices && navigator.mediaDevices.getUserMedia && window.MediaRecorde
}
async function startRealVoice(){
  let stream;
  try{
    stream = await navigator.mediaDevices.getUserMedia({audio:true});
  }catch(e){
    toast2('无法访问麦克风，改用话术选择');
    openSimSheet();
```

```
    return;
  }
  try{
    rec_ = (MediaRecorder.isTypeSupported && MediaRecorder.isTypeSupported('audio/webm'))
      ? new MediaRecorder(stream, {mimeType:'audio/webm'})
      : new MediaRecorder(stream);
  }catch(e){
    stream.getTracks().forEach(t=>t.stop());
    toast2('录音功能不可用，改用话术选择');
    openSimSheet();
    return;
  }
  const chunks = [];
  rec_.ondataavailable = e => { if(e.data && e.data.size) chunks.push(e.data); };
  rec_.onstop = async () => {
    stream.getTracks().forEach(t=>t.stop());
    const sec = Math.max(1, Math.round((Date.now()-recT0)/1000));
    const blob = new Blob(chunks, {type: rec_.mimeType || 'audio/webm'});
    if(!blob.size){ toast2('没有录到声音，请重试'); return; }
    try{
      const r = await AIBOOT.asr(blob);
      if(r && r.ok && r.text){
        /* 识别文本直接作答（与「话术选择」同一语义），语音条显示真实时长 */
        addMsg('out', r.text, {voice: sec});
        judge(r.text);
      }else{
        toast2('语音识别失败' + (r && r.error ? ': ' + r.error : '') + '，可切换键盘输入');
      }
    }catch(e){
      toast2('语音识别异常，可切换键盘输入');
    }
  };
  recT0 = Date.now();
  $('wave').innerHTML = Array.from({length:9},()=>'<i></i>').join('');
  Array.from($('wave').children).forEach((el,i)=>{ el.style.animationDelay=(i*0.09)+'s'; });
  $('recovText').textContent = '正在录音，点击结束';
  $('recov').classList.add('on');
  rec_.start();
}
function stopRealVoice(){
  if(rec_ && rec_.state === 'recording') rec_.stop();
  $('recov').classList.remove('on');
}
/* 轻量提示条（语音链路专用，不动已验收的提示区） */
let vtT = null;
/**
 * 结束本次练习（P27）
 *
 * 场景：学员练到一半有事要停；或演示时需要控制节奏。
 * 此前没有出口，只能刷新页面 —— 已练的轮次全部丢失（sessions 里有、records 里没有）。
```

```
 * 用「3 秒内点两次」防误触，不弹确认框（预览面板里 confirm 会被拦）。
 */
let endAskAt = 0;
function endPractice(){
  if (typeof running === 'undefined' || !running) {
    toast2('本次练习已结束，可直接查看报告');
    return;
  }
  if (Date.now() - endAskAt > 3000) {
    endAskAt = Date.now();
    toast2('再点一次右上角「…」结束练习并生成报告');
    return;
  }
  endAskAt = 0;
  if (typeof finish === 'function') finish();
}

function toast2(msg){
  let el = $('vtoast');
  if(!el){
    el = document.createElement('div');
    el.id = 'vtoast'; el.className = 'vtoast';
    document.body.appendChild(el);
  }
  el.textContent = msg; el.classList.add('on');
  clearTimeout(vtT); vtT = setTimeout(()=>el.classList.remove('on'), 2600);
}

/* 「按住说话」：真语音优先，模拟话术兜底 */
function openVoiceSheet(){
  if(!running || ended) return;
  if(voiceCapable() && window.AIBOOT && AIBOOT.asrAvail && AIBOOT.asrAvail()){
    startRealVoice();
    return;
  }
  openSimSheet();
}
/* 原话术选择（模拟语音识别结果）—降级路径原样保留 */
function openSimSheet(){
  const idx = Math.min(step, curScene().voicePool.length-1);
  const pool = curScene().voicePool[idx] || [];
  $('sheetBody').innerHTML = pool.map(t=>
    `<div class="opt" onclick='pickVoice(${JSON.stringify(t)})'>${esc(t)}<span class="lk">预计时长
  ).join('');
  $('sheet').classList.add('on');
}
function closeSheet(){ $('sheet').classList.remove('on'); }

/* 真机适配：窄屏浮层开关（任务/目标/操作说明收进 FAB） */
function toggleMobileInfo(){
```

```
  const sides = Array.from(document.querySelectorAll('.side'));
  const open = sides.some(s=>s.classList.contains('mopen'));
  sides.forEach(s=>s.classList.toggle('mopen', !open));
  const fab = document.getElementById('mobileFab');
  if(fab){ fab.classList.toggle('open', !open); fab.textContent = open ? '任务' : '收起'; }
}

function pickVoice(t){
  closeSheet();
  const dur = 4 + t.length % 9;
  $('wave').innerHTML = Array.from({length:9},()=>'<i></i>').join('');
  Array.from($('wave').children).forEach((el,i)=>{ el.style.animationDelay=(i*0.09)+'s'; });
  $('recov').classList.add('on');
  setTimeout(()=>{
    $('recov').classList.remove('on');
    addMsg('out', t, {voice:dur});
    judge(t);
  }, 1100);
}

function quickEmoji(){
  if(!running || ended) return;
  const v = $('msgInput');
  v.value += ['您好','谢谢','理解','好的'][Math.floor(Math.random()*4)];
  onInput();
}

/* ===================== 微信语音通话 ===================== */
function startCall(){
  if(!running || ended) return;
  inCall = true; callSec = 0; callTurns = 0;
  $('callAva').textContent = curScene().ava;
  $('callAva').style.background = curScene().color;
  $('callName').textContent = curScene().name;
  $('callState').textContent = '正在呼叫…';
  $('callTrans').textContent = '通话接通后，客户的话会实时转写在这里。';
  $('callTips').style.display='none';
  $('callScreen').classList.add('on');
  clearInterval(callTimer);

  setTimeout(()=>{
    if(!inCall) return;
    $('callState').textContent = fmt(0);
    callTimer = setInterval(()=>{
      callSec++;
      $('callState').textContent = fmt(callSec);
      if(callSec % 7 === 0 && callTurns < 3) callNextClient();
    }, 1000);
    callNextClient();
  }, 2000);
```

```
}

function fmt(s){ return String(Math.floor(s/60)).padStart(2,'0')+':'+String(s%60).padStart(2,'0'

function callNextClient(){
  const sc = curScene();
  const idx = Math.min(callTurns, sc.script.length-1);
  const d = sc.script[idx];
  // 通话中转写同样优先用服务端 NPC 的实时回复，消费一次即清空
  const useAI = !!(aiCustomerText && aiCustomerText.trim());
  const text = useAI ? aiCustomerText.trim() : d.t;
  aiCustomerText = null; aiVoiceSec = null;
  $('callTrans').innerHTML = `<span style="color:rgba(255,255,255,.45)">客户：</span>${esc(text)}
    + (useAI ? ' <span style="color:#8ee6ab;font-size:11px">AI 实时</span>' : '');
  $('callTips').style.display='block';
  const pool = sc.voicePool[idx] || [];
  $('callTipBody').innerHTML = pool.map((t,i)=>
    `<div class="o" onclick='callPick(${JSON.stringify(t)})'>${esc(t)}</div>`).join('');
  ttsSay(text);   // 通话中客户说话也要出声（P11 TTS）
}

function callPick(t){
  $('callTrans').innerHTML = `<span style="color:#8ee6ab">你：</span>${esc(t)}`;
  $('callTips').style.display='none';
  // 通话中的发言计入评分：命中加 13、通话加成 9，但**不计目标分**
  //  （与接入引擎前的行为严格一致：目标分只在文字/语音条路径由 judge 结算）
  if(window.SCORER){
    SCORER.scoreTurn(t, {
      state: dimState, kw: TRAIN.kw, dims: TRAIN.dims,
      inCall: true, noObjectiveBonus: true
    });
    TRAIN.dims.forEach((d,i)=>{ dimScore[i] = dimState[d.id]; });
  } else {
    DIMS.forEach((d,i)=>{ if(KW[d].some(k=>t.includes(k))) dimScore[i] = Math.min(97, dimScore[i
  }
  initDims();
  callTurns++;
  if(callTurns >= 3){
    setTimeout(()=>{ if(inCall){ hangUp(); } }, 1400);
  } else {
    setTimeout(()=>{ if(inCall) callNextClient(); }, 2400);
  }
}

function hangUp(){
  inCall = false;
  ttsStop();
  clearInterval(callTimer);
  $('callScreen').classList.remove('on');
  // 通话结束后，把通话内容记入聊天并出报告
```

```
  addMsg('out',' (语音通话已结束，通话时长 '+fmt(callSec)+') ');
  setTimeout(finish, 700);
}

/* 客户语音播报开关（P11 优化）：TTS 开关；不支持时如实提示，不假装可用 */
function toggleSpeaker(){
  if(!ttsSupported()){ toast2('当前浏览器不支持语音播报'); return; }
  ttsOn = !ttsOn;
  $('btnSpeaker').classList.toggle('mute-on', !ttsOn);
  if(!ttsOn) ttsStop();
  toast2(ttsOn ? '客户语音播报已开启（仅语音模式）' : '客户语音播报已关闭');
}
function toggleMute(){ $('btnMute').classList.toggle('mute-on'); }

/* ==================== 报告 ==================== */
/* 五维能力雷达（P14）：全部练习均分 vs 团队均分。
 * 口径走 TRAIN.helpers.dimAvgOf / overview().dimAvg，不在这里另算。
 * 为什么放在报告页：单场分数只说明这一场，能力画像要跨场看。 */
function radarMini(learnerId){
  const H = window.TRAIN && TRAIN.helpers;
  if(!H || !H.dimAvgOf) return '';
  const mine = H.dimAvgOf(learnerId);
  /* 无样本时 dimAvgOf 返回基准 42，画出雷达会被读成"我能力 42 分"——
     事实是"还没练过"，所以不画图。 */
  if(mine.every(d=>!d.n)){
    return '<div class="radar-empty">还没有练习记录，练完一场就会生成你的能力画像。</div>';
  }
  const team = (H.overview().dimAvg || []);
  const tmap = {};
  team.forEach(t=>{ tmap[t.id] = t.avg; });
  const dims = mine.map(d=>({ name:d.name, v:d.v, team:tmap[d.id]||0 }));
  const n = dims.length;
  if(!n) return '';
  const cx=150, cy=124, R=86;
  const ang = i => -Math.PI/2 + i*2*Math.PI/n;
  const pt = (i,v)=>{ const r=R*Math.max(0,Math.min(100,v))/100; return [cx+r*Math.cos(ang(i)),
  const poly = getter => dims.map((d,i)=>{ const p=pt(i,getter(d)); return p[0].toFixed(1)+','+p
  let g='';
  [25,50,75,100].forEach(lv=>{
    g += '<polygon points="'+poly(()=>lv)+'" fill="none" stroke="rgba(128,128,128,.35)" stroke-w
  });
  dims.forEach((_,i)=>{
    const p = pt(i,100);
    g += '<line x1="'+cx+'" y1="'+cy+'" x2="'+p[0].toFixed(1)+'" y2="'+p[1].toFixed(1)+'" stroke
  });
  g += '<polygon points="'+poly(d=>d.team)+'" fill="none" stroke="rgba(128,128,128,.65)" stroke-
  g += '<polygon points="'+poly(d=>d.v)+'" fill="rgba(62,181,117,.16)" stroke="currentColor" str
  dims.forEach((d,i)=>{
    const p = pt(i,d.v);
    g += '<circle cx="'+p[0].toFixed(1)+'" cy="'+p[1].toFixed(1)+'" r="2.6" fill="currentColor"/
```

```
    const lp = pt(i,120);
    const a2 = Math.abs(lp[0]-cx)<8 ? 'middle' : (lp[0]>cx ? 'start' : 'end');
    g += '<text x="'+lp[0].toFixed(1)+'" y="'+(lp[1]-2).toFixed(1)+'" text-anchor="'+a2+'" fill=
    g += '<text x="'+lp[0].toFixed(1)+'" y="'+(lp[1]+11).toFixed(1)+'" text-anchor="'+a2+'" fill
  });
  return '<div class="radar-wrap"><svg viewBox="0 0 300 250" width="100%" style="max-width:280px
    'role="img" aria-label="五维能力雷达：本人与团队均分对比">'+g+'</svg>' +
    '<div class="radar-lg"><b>实线</b> 我的全部练习均分  <b>虚线</b> 团队均分</div></div>';
}


function finish(){
  if(ended) return;
  ended = true; running = false;
  ttsStop();   // 报告页不该有客户声音残留
  // 综合得分：**AI 通道与服务端 finish 同口径**（各轮轮分平均），
  // 离线本地引擎沿用五维简单平均（本地引擎是「起始分+命中加分」的累加制，语义不同）
  const total = (aiMode === 'on' && dimRounds > 0)
    ? Math.round(scoreAccum / dimRounds)
    : Math.round(dimScore.reduce((a,b)=>a+b,0) / DIMS.length);
  const passLine = curTaskPass || curScene().pass;      // 任务可覆盖场景默认达标线
  const pass = total >= passLine;

  // 落库：会话维度已由服务端逐轮累计，这里结束会写入一条 Record（教练端/管理端可见）
  persistFinish(total, pass);

  // 任务来源标注：优先用本场绑定的任务，其次按场景找我的进行中任务，最后退回场景默认口径
  const feed = taskFeed();
  const _tk = (curTaskId && feed.filter(t=>t.id === curTaskId)[0])
    || feed.filter(t=>t.sceneId === curScene().id && t.status === 'running')[0];
  const taskLine = _tk
    ? `本任务：${_tk.title} · 由 ${_tk.createdBy || '培训运营'} 于 ${String(_tk.createdAt || '').s
    : '按场景默认达标线考核';

  const rows = DIMS.map((d,i)=>{
    const v = dimScore[i];
    const cls = v>=75?'g-good':(v>=60?'g-mid':'g-bad');
    const cmt = v>=75 ? '表现良好，继续保持。' : v>=60 ? '基本达标，还有提升空间。' : '明显偏弱，建议
    return `<div class="dm"><div class="t"><span>${d}</span><span>${v}</span></div>
      <div class="b"><i class="${cls}" style="width:${v}%"></i></div>
      <div class="cmt">${cmt}</div></div>`;
  }).join('');

  const items = reviewed.slice(0,6).map(r=>{
    /* 未命中时要说清"这轮要什么"（P21）。此前只有一句"未命中本轮考察点"，
       学员看完仍然不知道自己哪句话不对—评分变成数字而不是反馈。 */
    let why = r.feedback || '';
    if (!why) {
      if (r.ok) why = '命中本轮考察点，回答方向正确。';
      else {
```

```
        const di = r.objDim ? DIMS.findIndex(d => d.id === r.objDim) : -1;
        const dn = di >= 0 ? DIMS[di].name : '';
        const dv = di >= 0 ? dimScore[di] : null;
        why = '这轮要的是「' + (r.objText || '本轮目标') + '」' +
          (dn ? '，对应能力「' + dn + '」（本场 ' + dv + ' 分）' : '') +
          '——你的回答没有落在这个点上。';
      }
    }
    return `<div class="rep-item">
      <div class="q">客户：${esc(r.q)}</div>
      <div class="a ${r.ok?'good':'bad'}"><div class="dot"></div><div class="tx">你：${esc(r.a)}<
        <span style="color:#8A8A8A">${esc(why)}</span></div></div>
    </div>`;
  }).join('') || '<div class="rep-item" style="color:#8A8A8A">练习记录不足，建议重新完整走一遍。</d

  /* 和上次相比（P24）
   * 学员最想知道"我进步了吗"。口径：**同一场景**的上一场（不同场景难度不同，混着比不公平）；
   * 第一次练这个场景就没有对比块，而不是拿"历史均分"硬凑一个数。 */
  const prevSame = (window.TRAIN && TRAIN.records ? TRAIN.records : [])
    .filter(r => r.learnerId === ME && r.sceneId === curScene().id)
    .sort((a, b) => (String(a.finishedAt) < String(b.finishedAt) ? -1 : 1))
    .pop();
  const cmpHtml = (function () {
    if (!prevSame) return '';
    const cmpCls = (d) => (d > 2 ? 'up' : (d < -2 ? 'down' : 'flat'));
    const cmpTxt = (d) => (Math.abs(d) <= 2 ? '持平' : (d > 0 ? '+' : '') + d);
    /* ⚠ 学员端的 DIMS 是「维度名字符串数组」，不是对象数组（helpers 里那份才是对象）——
       所以不能取 d.id，要按索引映射到 d1..d5。 */
    const DIM_IDS = ['d1', 'd2', 'd3', 'd4', 'd5'];
    const dimRows = DIMS.map(function (d, i) {
      const before = (prevSame.dimScores || {})[DIM_IDS[i]];
      if (typeof before !== 'number') return '';
      const diff = dimScore[i] - before;
      return '<div class="cmp-row"><span>' + esc(d) + '</span><b>' + dimScore[i] +
        '</b><i class="' + cmpCls(diff) + '">' + cmpTxt(diff) + '</i></div>';
    }).join('');
    if (!dimRows) return '';
    const dt = total - (prevSame.score || 0);
    return '<div class="rcard"><h3>和上次相比' +
      '<small style="font-weight:400;color:#8A8A8A;font-size:11px;margin-left:6px">同场景 · ' +
      esc(String(prevSame.finishedAt || '').slice(5, 10)) + '</small></h3>' +
      '<div class="cmp-row cmp-total"><span>综合</span><b>' + total + '</b><i class="' +
      cmpCls(dt) + '">' + cmpTxt(dt) + '</i></div>' + dimRows + '</div>';
  })();

  // AI 通道说明条：让学员/评审一眼看出本场是真实模型评分还是降级
  /* 评分通道如实说明（P26）：mock 不能写成"AI 评分"，否则被追问模型名时无法自圆 */
  const srcLine = (function () {
    if (aiMode !== 'on') return '本场为 <b>本地关键词引擎</b>评分（离线模式），未连接评分服务';
    if (aiChannel === 'llm') {
```

```
      return '本场为 <b>AI 评分</b>' + (aiModelName ? ' (模型 ' + aiModelName + ') ' : '') +
        (aiLastTurn && aiLastTurn.confidence != null
          ? ', 置信度 ' + (aiLastTurn.confidence * 100).toFixed(0) + '%' : '');
    }
    if (aiChannel === 'mock') {
      return '本场为 <b>模拟评分通道</b>（本地 mock，用于演示链路），非真实模型评分';
    }
    return '本场为 <b>服务端关键词引擎</b>评分（未配置模型 Key）';
  })();

  $('repBody').innerHTML = `
    <div class="rcard score-hero">
      <div class="num" style="color:${pass?'#07C160':'#FA5151'}">${total}<small>分</small></div>
      <div class="lbl">综合得分 · 本次达标线 ${passLine} 分</div>
      <div class="lbl" style="margin-top:6px;font-size:12px">${taskLine}</div>
      <div class="verdict ${pass?'v-pass':'v-fail'}">${pass?'已达标':'未达标，建议再练一次'}</div>
      <div class="src-note">${srcLine}</div>
    </div>
    <div class="rcard"><h3>能力维度 · 本场</h3>${rows}
      <div style="margin-top:12px;padding-top:10px;border-top:1px solid rgba(128,128,128,.18)">
        <div style="font-size:12px;color:#8A8A8A;margin-bottom:4px">能力画像（全部练习均分 · 虚线为
        ${radarMini(ME)}
      </div>
    </div>
    ${cmpHtml}
    <div class="rcard"><h3>关键对话点评</h3>${items}</div>
    <div class="rcard"><h3>AI 带教建议</h3>
      <div style="font-size:13px;line-height:1.8;color:#4A4A4A">
        本次练习中，<b>${DIMS[dimScore.indexOf(Math.min(...dimScore))]}</b> 是相对薄弱的环节。
        建议下一轮练习时，先在心里走一遍目标清单再开口；遇到客户压价或质疑时，先把问题问清楚，再给方案。
        带教主管会在 24 小时内看到这份报告，并安排一次一对一辅导。
      </div>
    </div>`;
  $('reportScreen').classList.add('on');
  try { renderNextTaskBtn(); } catch (e) {}
}

/* 结束会话 → 服务端落库（失败不影响报告展示） */
async function persistFinish(total, pass){
  if(aiMode !== 'on' || !window.AIBOOT || !aiSessionId) return;
  try{
    const r = await AIBOOT.finish({ sessionId: aiSessionId });
    if(r.ok){
      console.log('[AI] 已落库 Record: ' + (r.data && r.data.recordId || ''));
      aiRecordId = r.data && r.data.recordId || null;
    } else {
      console.warn('[AI] 落库失败: ' + r.error);
    }
  }catch(e){ console.warn('[AI] 落库异常: ' + e.message); }
}
```

```
2851
2852 /* ==================== 其他 ==================== */
2853 /* 手机内部主题（与「页面外壳主题」互相独立，各切各的） */
2854 function toggleTheme(){
2855   const s = $('screen');
2856   s.dataset.theme = s.dataset.theme === 'dark' ? '' : 'dark';
2857 }
2858 /* 页面外壳主题：与 index / 管理后台 / 教练台 / PRD 共用 train-theme 这个键 */
2859 function syncThemeBtn(){
2860   const b = $('btnPageTheme');
2861   if(b) b.textContent = document.documentElement.dataset.theme === 'light' ? '深色模式' : '浅色模:
2862 }
2863 function togglePageTheme(){
2864   document.documentElement.dataset.theme =
2865     document.documentElement.dataset.theme === 'light' ? 'dark' : 'light';
2866   try{ localStorage.setItem('train-theme', document.documentElement.dataset.theme); }catch(e){}
2867   syncThemeBtn();
2868 }
2869 function nowTime(){
2870   const d = new Date();
2871   return String(d.getHours()).padStart(2,'0')+':'+String(d.getMinutes()).padStart(2,'0');
2872 }
2873 function tick(){
2874   const t = nowTime();
2875   $('clock').textContent = t;
2876   $('clock2').textContent = t;
2877 }
2878
2879 /* 事件委托：滑动查看语音条提示 */
2880 document.addEventListener('click', e=>{
2881   /* 我的任务 → 开始训练（绑定该任务，成绩才能归到任务进度里） */
2882   const st = e.target.closest('[data-start-task]');
2883   if(st && !st.disabled){ startTask(st.getAttribute('data-start-task')); return; }
2884
2885   const b = e.target.closest('.bub.voice');
2886   if(b && !b.dataset.played){
2887     b.dataset.played = '1';
2888     b.style.opacity = '.72';
2889     setTimeout(()=>{ b.style.opacity = '1'; b.dataset.played=''; }, 900);
2890   }
2891 });
2892
2893 /* ==================== 演示直达 ====================
2894    在地址后加 ?demo=chat / ?demo=call / ?demo=report 可直达对应界面，
2895    方便截图、评审和录屏演示。                                          */
2896 function boot(){
2897   const m = new URLSearchParams(location.search).get('demo');
2898   if(!m) return;
2899   DEMO = true;
2900   if(m==='call'){
```

```
    openChat(0);
    setTimeout(()=>startCall(), 80);
    return;
  }
  if(m==='report'){
    openChat(0);
    setTimeout(()=>{
      dimScore=[88,76,63,71,55];
      objDone=[true,true,false,true,false];
      reviewed=[
        {q:SCENES[0].script[0].t, a:'经理您好，我是示例食品的小张，上次给您发的报价单您看到啦？想先了解
        {q:SCENES[0].script[1].t, a:'您一年用量大概在什么量级？我按量重新给您算一版整体成本。', ok:tr
        {q:SCENES[0].script[2].t, a:'理解您的担心。上个月我们刚给同城另一家连锁供了一批，可以安排您去看
        {q:SCENES[0].script[3].t, a:'售后我们承诺同城4小时到场，出问题先补货、后对账。', ok:true},
        {q:SCENES[0].script[4].t, a:'我下午5点前把方案和报价一起发到您微信上，明天上午跟您确认一下领导
      ];
      initDims(); renderObjs(); finish();
    }, 80);
    return;
  }
  // 默认 chat
  openChat(0);
  const sc = SCENES[0];
  setTimeout(()=>{
    $('chatBody').innerHTML = `<div class="time-div">今天 ${nowTime()}</div>`;
    addMsg('in', sc.script[0].t, {voice:sc.script[0].v});
    addMsg('out','经理您好，我是示例食品的小张，上次给您发的报价单您看到啦？');
    addMsg('in', sc.script[1].t, {voice:sc.script[1].v});
    addMsg('out','您方便的话，我想先了解一下，您主要是觉得价格有压力，还是对品质有顾虑？');
    addMsg('in', sc.script[2].t, {voice:sc.script[2].v});
    dimScore=[86,79,64,71,58]; initDims();
    objDone=[true,true,false,false,false]; renderObjs();
    $('tipBox').innerHTML = sc.tips[2];
    $('msgInput').value='样品我们可以先寄两份过来，您把收货地址发我一下？'; onInput();
  }, 80);
}

/* 页面外壳主题：恢复上次选择 */
try{
  const savedTheme = localStorage.getItem('train-theme');
  if(savedTheme === 'light' || savedTheme === 'dark') document.documentElement.dataset.theme = s
}catch(e){}
syncThemeBtn();

initList(); renderTasks(); initDims(); tick(); setInterval(tick, 15000);
/* 服务端在线时同步「我的任务」（新下发的任务才收得到）；失败静默走本地 */
loadMyTasks();
/* 服务端在线时同步「已发布场景池」（运营新发布的场景学员才拿得到）；失败静默走本地种子 */
loadScenes();
/* 轻轮询：页面开着也能收到新发布的场景与新下发的任务（不必刷新）。
```

```
    只在页面可见时拉（后台标签页不耗流量也不打服务端）；
    loadScenes/loadMyTasks 内部都是「有变化才重渲染 + 失败静默」，轮询不会抖动界面。 */
setInterval(function(){
  if(document.visibilityState === 'visible'){ loadScenes(); loadMyTasks(); }
}, 60000);

/* 无头验证探针（只读，不影响页面行为） */
window.__LRN = () => ({
  me: ME,
  meName: myName(),
  serverTasks: serverTasks ? serverTasks.length : null,
  myTasks: taskFeed().map(t => t.id + ':' + t.status),
  myTaskScenes: taskFeed().filter(t => t.status === 'running').map(t => t.sceneId),
  curTaskId: curTaskId,
  curTaskPass: curTaskPass,
  sceneIds: SCENES.map(s => s.id),
  sceneCount: SCENES.length
});
/* Esc 关闭（P14）：先有关闭，再谈快捷——被浮层挡住时不用去找关闭按钮 */
document.addEventListener('keydown', function (e) {
  if (e.key !== 'Escape') return;
  var guide = document.getElementById('guide');
  if (guide && guide.classList.contains('on')) { closeGuide(); return; }
  if (stopRealVoice && document.getElementById('recov') && document.getElementById('recov').clas
  var sheet = document.getElementById('sheet');
  if (sheet && sheet.classList.contains('on')) { closeSheet(); return; }
});

window.addEventListener('resize', scrollBottom);
boot();
/* 首屏任务卡（P15）：等首次任务/成绩同步落地后填一次，避免整卡"–" */
setTimeout(function(){ try{ renderTaskCard(null); }catch(e){} }, 1200);
setTimeout(function(){ try{ renderCoachNotes(); }catch(e){} }, 1400);
setTimeout(function(){ try{ renderGrowth(); }catch(e){} }, 1500);
setTimeout(function(){ try{ renderHistory(); }catch(e){} }, 1600);

/* 首次引导（P12）：新学员第一次打开给三步说明；点任意按钮写标记，之后不再出现 */
function showGuideIfFirst(){
  try{ if(localStorage.getItem('train-guide-seen') === '1') return; }catch(e){ return; }
  const el = document.getElementById('guide');
  if(el) el.classList.add('on');
}
function closeGuide(){
  const el = document.getElementById('guide');
  if(el) el.classList.remove('on');
  try{ localStorage.setItem('train-guide-seen', '1'); }catch(e){ /* 隐私模式下写不了，本次仍可关闭
}
setTimeout(showGuideIfFirst, 700);
```